

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE

CORSO DI LAUREA IN INFORMATICA

Tesi di laurea triennale

BlueAgent

2025-2026

CANDIDATO:

Davide Luci

Matricola 0341922

RELATORE:

Pierpaolo Loreti

CORRELATORE:

Ing. Francesco Mancini

Indice

1 Introduzione **1**

2 Contesto e Confronto con lo Stato dell'Arte **3**
2.1 CR Alternativi: kCTF 3
2.2 CR Alternativi: CTFBox 7
2.3 CR Alternativi: CyTrONE 9
2.3.1 Valutazione delle Performance di CyTrONE 12
2.4 CR Alternativi: CyberRangeCZ 14
2.5 CR Alternativi: CyberPatriot 18

3 BlueAgent **19**
3.1 Attori 20
3.1.1 CTF Manager (attore) 20
3.1.2 Giocatore (attore esterno per CTF Manager) 21
3.1.3 Autore della sfida (attore esterno per CTF Manager) 21
3.1.4 Amministratore della competizione (attore esterno per CTF Manager) 21
3.2 Use Cases per CTF Manager 22
3.2.1 `UC-01`: Deploy dell'infrastruttura di rete per il team k 22
3.2.2 `UC-02`: Deploy della challenge per il team k 23
3.2.3 `UC-03`: Verifica della challenge per il team k 24
3.2.4 `UC-04`: Dismissione e rilascio di una challenge per il team k . . 24
3.2.5 `UC-05`: Compilazione e Import di un'immagine per container . 25
3.2.6 `UC-06`: Registrazione e configurazione WireGuard per un membro del team k 26
3.3 Requisiti di Sistema 27
3.3.1 Piattaforma Host e Infrastruttura 27
3.3.2 Motore di Containerizzazione 27
3.3.3 Rete Ibrida e Connettività 28

3.3.4 Motore di Verifica OVN Sandbox 29
3.3.5 Control Plane, Concorrenza e Persistenza 29
3.3.6 Pipeline Immagini Distrobuilder 30
3.4 Meccanismi di Verifica ed Oracoli 31
3.4.1 Verifica Statica 31
3.4.2 Verifica Dinamica tramite orchestratore esterno 32
3.4.3 Verifica Dinamica tramite injection dei comandi di verifica . . 33
3.4.4 Clonazione del sistema in una rete separata 33

4 Architettura di Rete ad Alto Livello 34
4.1 Rete production-*k* 34
4.2 Rete verify-*k* 35
4.3 Struttura e configurazione di una Challenge 39
4.3.1 Struttura del File di Configurazione della Challenge 40
4.3.2 Configurazione delle Immagini con Distrobuilder 42

5 Architettura nel Dettaglio e Validazione 43
5.1 Architettura del Codice 43
5.1.1 Astrazione del Runtime: `ContainerEngine` 43
5.1.2 Ciclo di Vita della Sandbox: Async Context Manager (`OVNVerifier`) 44
5.1.3 Concorrenza e Two-Tier Locking 45
5.1.4 Pipeline Asincrona non Bloccante su Socket Unix 46
5.1.5 Resilienza e Riconciliazione dello Stato (`LifecycleManager`) . . 46
5.1.6 Compilazione delle immagini 46
5.2 Comandi per l'Architettura di Rete 47
5.2.1 Configurazione OVN e Open vSwitch 47
5.2.2 production-*k* Routing e Connettività dei Peer (WireGuard e nftables) 47
5.3 Plugin CTFd 49
5.3.1 Componenti del Plugin: PatchChallenge e PatchFlag 49
5.3.2 Distribuzione delle Configurazioni WireGuard 49
5.3.3 Flusso di Verifica e Convalida 50
5.4 Validazione 50
5.4.1 Confronto tra Alpine e Ubuntu 52
5.4.2 Confronto tra Clonazione e Deploy 53
5.4.3 Uso di Memoria 55

6 Architetture di Rete Alternative 56

6.1 Bridge L2 Senza Gateway 56
6.2 Interfaccia wan1 Aggiuntiva 57
6.3 Network Namespace Separato per la Verifica 57
6.4 Router Intermedio 58
6.5 Tabella riassuntiva 59

7 Conclusioni e Sviluppi Futuri **60**
7.1 Difesa e Validazione di BlueAgent 60
7.2 Compromessi e Sviluppi Futuri 61
7.2.1 Compromessi Architetturali 61
7.2.2 Sviluppi Futuri 62

8 Appendice **63**
8.1 Linux Networking 63
8.1.1 Network Device Driver 63
8.1.2 Bridge 64
8.1.3 Netfilter 65
8.2 WireGuard 66
8.3 ZFS 68
8.3.1 Vantaggi di Pool e transazioni 69
8.3.2 Considerazioni di Efficienza e Frammentazione 70
8.4 NsJail e Container 70
8.4.1 Namespaces 70
8.4.2 Seccomp 71
8.4.3 cgroups 71

Bibliografia **73**

Ringraziamenti **76**

Capitolo 1

Introduzione

Nel lavoro di tesi vogliamo gestire una competizione CTF (Capture The Flag).

Di solito in una competizione CTF, uno o più utenti (eventualmente organizzati in team) si sfidano per risolvere un insieme di challenge, ossia recuperare una **flag**: una stringa del tipo `{hai_trovato_la_flag}`.

Le CTF hanno lo scopo di migliorare la formazione degli specialisti del settore, tramite la simulazione di scenari realistici, complessi o quantomeno molto specifici. In particolare è essenziale formare studenti e risorse aziendali per la gestione e mitigazione di sistemi vulnerabili, in modo da prevenire perdite dovute ad attacchi informatici in scenari reali. La simulazione di questi scenari avviene all'interno di un ambiente denominato Cyber Range.

Ci concentreremo nel lavoro in un tipo di competizione CTF, basato sulla gestione di uno o più sistemi vulnerabili da mitigare, dunque una competizione di Difesa dove la flag si ottiene unicamente difendendo il sistema, ossia *"patchando"* la vulnerabilità. Nel lavoro si tiene conto di due requisiti da soddisfare:

- Il sistema vulnerabile spesso potrebbe essere costituito da uno o più host/nodi che comunicano tra di loro.
- Il Cyber Range dovrebbe essere in grado di determinare in autonomia e senza interferire con i giocatori, lo stato della macchina.

Per soddisfare tali requisiti abbiamo sviluppato *BlueAgent*, un orchestratore basato sulla tecnologia dei container che implementa il Cyber Range di cui abbiamo bisogno per gestire competizioni di Difesa. BlueAgent si occupa di verificare che le vulnerabilità presenti sui container siano state corrette, tramite un sistema di verifica che avviene completamente all'oscuro dei giocatori, ossia la clonazione dei container in un ambiente separato.

Per valutare l'efficacia della soluzione proposta, l'architettura è stata validata misurando i tempi di esecuzione e assorbimento delle risorse (CPU e memoria RAM) al crescere del numero dei team e dei container istanziati.

Capitolo 2

Contesto e Confronto con lo Stato dell'Arte

Secondo il NIST, un **Cyber-Range** (CR) è definito nel seguente modo:

> "Un ambiente interattivo, che simula la rete locale, i sistemi, i tool e le applicazioni proprietarie di un'azienda o di un'organizzazione". [1]

Illustreremo nelle sezioni successive alcune piattaforme per la gestione di Cyber Range, riassumendo brevemente le differenze di ognuno con BlueAgent.

2.1 CR Alternativi: kCTF

kCTF[2] è una piattaforma sviluppata da Google basata su *Kubernetes* e *nsjail*.

> *Kubernetes*: è un sistema open source per automatizzare il deploy, scalare e gestire applicazioni containerizzate.[3]

> *NSJail* NameSpace Jail. È un tool Linux per isolare i processi utilizzando varie feature del kernel come: namespaces, resource limits e seccomp-bpf syscall filters. L'obiettivo di NSJail è quello di creare un ambiente sicuro e isolato per eseguire le applicazioni. [4]

Facendo riferimento alla Figura 2.1, l'architettura di kCTF prevede:

[1] https://www.mdpi.com/1424-8220/20/24/7148 1. Introduction

[2] Google Security Team, *kCTF: Kubernetes-based CTF infrastructure*, https://google.github.io/kctf/

[3] The Linux Foundation, *Kubernetes Documentation*, https://kubernetes.io/docs/

[4] Google, *NsJail: A light-weight process isolation tool*, https://github.com/google/nsjail/

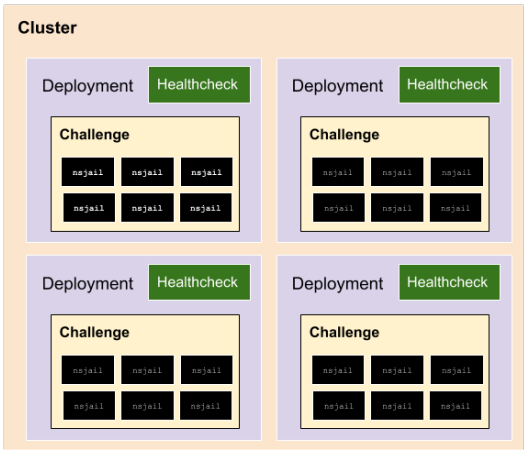


Figura 2.1: Architettura di kCTF basata su Kubernetes e NsJail

- Un **cluster**, che contiene un insieme di challenge
- Dei **deployment**, ognuno contiene due container (o meglio, un Pod che istanzia 2 container):
  - un container `healthcheck`: usato per determinare lo stato di salute dei servizi
  - un container `challenge`: contiene i servizi su cui lavora lo sfidante.

  Pod: in Kubernetes, è la più piccola unità che Kubernetes può eseguire. Un Pod, rappresenta uno o più container in esecuzione su un singolo nodo.[5]

  Deployment: in Kubernetes, un deployment è un template che specifica come creare i container. I template guidano la creazione di **Pods** e la gestione sicura del ciclo di vita di un Pod, in particolare la gestione dei ReplicaSet.[5]

NSJail viene avviato su ogni container `challenge`, ed è configurato per ascoltare su una porta TCP nota. Ad ogni connessione TCP in arrivo, NSJail esegue un fork restituendo allo sfidante un ambiente separato (rispetto agli altri sfidanti) su cui lavorare. L'obiettivo di questa architettura è di poter riutilizzare lo stesso container per più sfidanti.

L'isolamento fornito da NSJail, si ottiene combinando varie tecniche:

- Creazione di nuovi namespace: l'ambiente fornito, esegue su un nuovo namespace, separando logicamente le risorse accessibili.

[5]The Kubernetes Authors, *Workload Management: Pods and Deployments*, https://kubernetes.io/docs/concepts/workloads/

- Filtraggio delle chiamate di sistema tramite `seccomp-bpf`: questo sottosistema permette di uccidere i processi nell'ambiente se vengono eseguite chiamate di sistema vietate.
- Accounting e limitazione delle risorse con `cgroup`: questo sottosistema permette di monitorare e limitare le risorse utilizzabili nel nuovo ambiente.

NsJail può essere configurato in modo che ogni ambiente isolato, abbia a disposizione un filesystem temporaneo che risiede in RAM, mentre il filesystem originale è accessibile dall'ambiente sandbox in sola lettura. Questa configurazione è necessaria poiché altrimenti gli sfidanti modificherebbero tutti lo stesso filesystem.

Un'altra configurazione di NSJail è la seguente, ideale per le challenge di tipo `pwn` (trovare e sfruttare vulnerabilità in file binari):

```
[...]
mount: [
  {
    src: "/chroot"
    dst: "/"
    is_bind: true
  },
  {
    dst: "/tmp"
    fstype: "tmpfs"
    rw: true
  },
  {
    dst: "/proc"
    fstype: "proc"
    rw: true
  },
  {
    src: "/etc/resolv.conf"
    dst: "/etc/resolv.conf"
    is_bind: true
  }
]
```

1. La cartella `/chroot` che risiede nel container, diventa la radice nel nuovo ambiente.
2. Viene creato un nuovo `tmpfs`, in `/tmp`
3. Viene importata la configurazione DNS da `/etc/resolv.conf` dentro la nuova radice.

Tuttavia, l'uso di NsJail per separare gli ambienti dei partecipanti, rende difficoltosa l'implementazione di challenge in cui si vuole dare la libertà ai partecipanti

di installare pacchetti dentro al container, oppure di modificare file di configurazione che risiedono per esempio in `/etc` e di modificare direttamente servizi in esecuzione sul container. [6]

Queste limitazioni ci sono perché NsJail nell'ambiente sandbox non fornisce una propria istanza di `systemd` (o un qualsiasi gestore di servizi) e neanche l'infrastruttura necessaria per eseguire comandi come `apt`.

NsJail porta con sé altri problemi:

- Se dovesse cadere la connessione TCP allora tutto il contenuto del filesystem in RAM andrebbe perso.
- L'uso di `seccomp` per filtrare le chiamate di sistema potrebbe impedire l'esecuzione di alcuni programmi essenziali per la challenge.

Dunque kCTF potrebbe essere utilizzato per implementare un Cyber Range difensivo ma con varie limitazioni e problemi da aggirare.

L'approccio di kCTF è completamente all'opposto di quanto fatto in BlueAgent ed in altri Cyber Range. BlueAgent si occupa di gestire più container per ogni team, mentre kCTF assegna un container per tutti i team da dividere usando ambienti separati con NsJail.

Caratteristica	kCTF	BlueAgent
Virtualizzazione	Container applicativi e sandbox di processo (NSJail)	Container di sistema, ognuno con un suo gestore di servizi.
Infrastruttura	Cluster distribuito Kubernetes	Singolo nodo host
Ambiente Dedicato	Un solo container condiviso tra sfidanti	Container di sistema dedicati e separati per ciascun team
Persistenza	Filesystem effimero in RAM (`tmpfs`)	Persistente su disco
Provisioning	Istantaneo (consiste nel fork con NsJail)	Nell'ordine dei secondi (deploy di uno o più container)

[6] Google Security Team, *kCTF Security and Threat Model*, https://github.com/google/kctf/blob/v1/docs/security-threat-model.md

2.2 CR Alternativi: CTFBox

CTFBox è un Cyber Range progettato per organizzare CTF di tipo Attack/Defense, dove ogni team ha una macchina con uno o più servizi vulnerabili da difendere, mentre si attaccano i servizi esposti sulle macchine avversarie per recuperare le flag. Durante il gioco, ogni team deve fare attenzione a non rompere i propri servizi, mantenendo un'alta percentuale di SLA. Una SLA bassa comporta una penalità in termini di punteggio. [7]

Fa uso di **Docker** per gestire i servizi essenziali del gioco (e che costituiscono quindi CTFBox), mentre **Incus** viene utilizzato per gestire l'esecuzione delle macchine virtuali o dei container dei team (Figura 2.2).

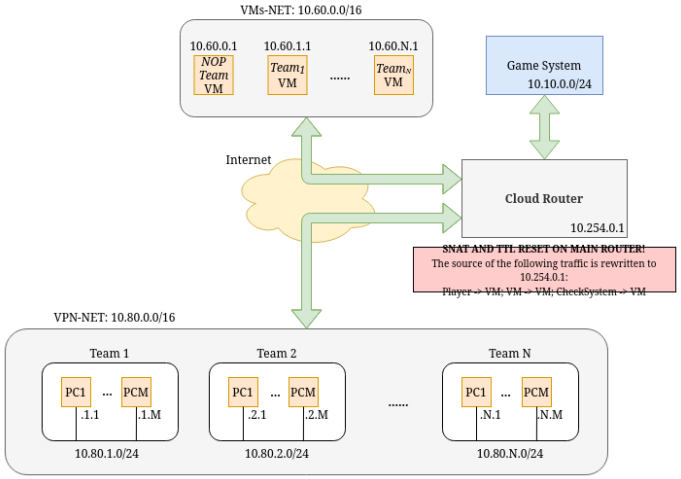


Figura 2.2: Architettura modulare di CTFBox

Router: è un container Docker, che gestisce le configurazioni della rete WireGuard, permettendo ai giocatori di connettersi alla rete. Controlla le regole di routing sulla rete, e governa la transizione della rete tra 3 stati diversi:

- `freeze`: in questo stato nessuno può accedere alle macchine vulnerabili
- `lock`: in questo stato ogni team può accedere solo alla propria macchina vulnerabile.
- `unlock`: in questo stato la rete è completamente aperta, i team possono attaccare le macchine avversarie.

Game System: è un container Docker, che agisce da componente centrale per la gestione del gioco.

[7] Domy, *CTFBox: Automated CTF Infrastructure Framework*, https://ctfbox.domy.sh/

- Gestisce lo stato della rete (freeze/lock/unlock), comunicando i cambiamenti al router.
- Mantiene il database delle flag, e calcola i punteggi per ogni team.
- Coordina i Checkers, ossia i worker che interrogano i servizi dei team ad ogni tick.

In CTFBox, ad ogni tick (ossia ogni round di gioco), le flag vengono rigenerate su ogni macchina, per ogni servizio vulnerabile, e poi registrate nel database. L'obiettivo di un team in una competizione Attack/Defense strutturata in questo modo è di attaccare più volte lo stesso servizio sulle macchine avversarie, per recuperare quante più flag possibili, finché le difese avversarie non bloccano gli attacchi.

I **Checkers** fanno due cose ad ogni tick:

1. Interrogano tutti i servizi vulnerabili attivi, e determinano la percentuale di SLA per ogni team.
2. Installano le nuove flag sui servizi vulnerabili.

Ogni membro del team, ha la sua configurazione **WireGuard** per connettersi alla propria macchina e per raggiungere le macchine avversarie. Ad ogni giocatore che si connette viene assegnato un indirizzo nella sottorete `10.80.team_id.0/24` mentre le macchine dei team si trovano nella sottorete `10.60.0.0/16`.

Dunque CTFBox si rivela essere un ottimo sistema per gestire competizioni Attack/Defense e per regolare l'accesso alle macchine, ma la sua architettura non si applica in contesti in cui si desidera assegnare ad ogni team un insieme di macchine vulnerabili il cui stato verrà poi verificato da un oracolo.

Sia CTFBox che BlueAgent usano WireGuard ed Incus per gestire le connessioni dei partecipanti e le macchine dei vari team.

Tuttavia CTFBox gestisce intere macchine virtuali mentre BlueAgent per design gestisce di default solo container di sistema.

Caratteristica	CTFBox	BlueAgent
Modello di Gioco	Attack & Defense competitivo tra team	Defensive / Blue Teaming & Patching
Virtualizzazione	Macchine Virtuali / Container eterogenei	Container (Incus)
Topologia di Rete	Subnet condivisa aperta (`10.60.0.0/16`)	LAN virtuale dedicata e isolata per team

Caratteristica	CTFBox	BlueAgent
Meccanismo di Verifica	Polling periodico dello stato dei servizi su rete di gioco (Checkers a tick, assenza di un Oracolo)	Esecuzione di un Oracolo su cloni in rete OVN isolata
Invisibilità della Verifica	Assente (inoltre, il traffico dei checker viaggia su rete condivisa)	Totale (l'oracolo esegue in una rete SDN separata)
Calcolo del Punteggio	SLA di uptime dei servizi e flag rubate	SLA non previsto.

2.3 CR Alternativi: CyTrONE

CyTrONE[8] è un framework incentrato sul training che integra contemporaneamente sia una piattaforma educativa che un Cyber Range.

La seguente immagine (Figura 2.3) mostra i componenti di CyTrONE:

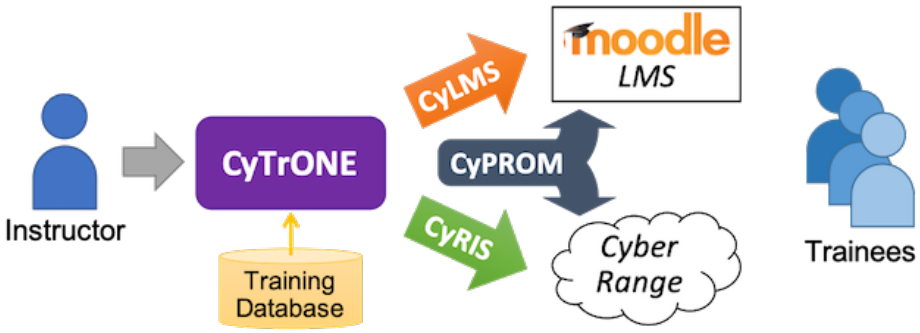


Figura 2.3: Architettura generale del framework CyTrONE

- **CyLMS**: si occupa di caricare il contenuto per il training su un Learning Management System come **Moodle**
- **CyRIS** (Cyber Range Instantiation System): si occupa di istanziare l'ambiente di training su cui gli studenti possono condurre opportuni test per avanzare nel training.
- **CyPROM**: si occupa di tracciare l'avanzamento controllando LMS ed il Cyber Range.

[8] https://github.com/crond-jaist/cytrone

CyTrONE gestisce tre tipi di training[9]:

- Training *Attack-oriented*: si basa sul capire come riprodurre attacchi a vulnerabilità note.
- Training *Analysis-oriented*: si concentra sull'analisi dell'attacco, per capire come funziona e le sue conseguenze.
- Training **Defense-oriented**: si concentra sulle tecniche di patching delle vulnerabilità, al fine di proteggere il sistema.

Quindi l'esperienza utente si concentra sul fatto che gli studenti possono eseguire un training e visualizzare le risorse associate sul Moodle ed al tempo stesso accedere al Cyber Range associato.

CyTrONE si applica bene facendo le seguenti assunzioni:

1) *L'obiettivo di ciascuna attività di formazione può essere formulato come un problema, che può dipendere o meno da altri problemi della stessa lezione (es: trova l'indirizzo IP, trova la flag);* [9]

2) *Il sistema è in grado di stabilire automaticamente se il partecipante abbia risolto il problema, confrontando la risposta fornita con un valore noto* [9]

L'architettura di CyTrONE è supportata da un *Training Database*, che si occupa di tenere in memoria le informazioni per i vari training, che vengono caricate su CyTrONE da una figura identificata come *Training Coordinator*. Le informazioni mantenute nel database sono per esempio: il training content, file e programmi, macchine virtuali, ecc... [9]

[9]https://www.sciencedirect.com/science/article/pii/S0167404818306527

CyTrONE utilizza un file YAML per definire il contenuto didattico di un training:

```yaml
training:
  - id: "L1-EN"
    title: "Analisi di Sicurezza su Host Desktop"
    overview: "Scenario di indagine su potenziale intrusione nel sistema aziendale."
    level: 1
    questions:
      - id: "Q-001"
        body: "Individuare la versione esatta del kernel Linux in esecuzione."
        answer: "3.10.0-514.21.1.el7.x86_64"
        hints:
          - "Ispezionare /proc/version oppure eseguire 'uname -r'."
```

E similmente, si usa un file YAML per definire la specifica dell'infrastruttura Cyber Range:

```yaml
host_settings:
  - id: "host_1"
    virbr_addr: "192.168.122.1/24"

guest_settings:
  - id: "desktop"
    basevm_type: "kvm"
    tasks:
      - add_account: { account: "analyst", passwd: "analyst_password" }
      - install_package: { manager: "yum", name: "wireshark" }
      - emulate_attack: { type: "ssh_bruteforce", target: "analyst" }
      - emulate_malware: { name: "miner_daemon", cpu_utilization: 40 }
```

A partire dalla configurazione YAML del Cyber Range, il sotto-sistema CyRIS si occupa di istanziarlo. Successivamente, in caso di un training **Defense-oriented** l'utente richiede l'attacco associato alla vulnerabilità all'interno del sistema e ne viene registrato l'esito.

Il sistema ha le seguenti caratteristiche:

1. Il deploy delle vulnerabilità avviene su macchine virtuali
2. Il payload viene inviato remotamente dal `Cyberattack Controller` (basato sull'esecuzione di `Metasploit`).
3. Necessita di un LMS (per design) per tenere traccia del training progress.

2.3.1 Valutazione delle Performance di CyTrONE

Le performance di CyTrONE per il deploy dell'infrastruttura sono state valutate usando il testbed StarBED [10], usando 30 host fisici e 600 macchine virtuali. Gli host fisici usati per misurare le performance sono dei *server rack Cisco UCS C200 M2 con Intel Xeon E5504 2.0 GHz a 4 core e 72 GB di memoria*

Sono stati valutati 2 scenari:

- *Level 1*: un training base che include tra gli argomenti: log, configurazioni di sistema, sniffing della rete e scansione delle vulnerabilità. Richiede un'unica macchina Virtuale per fare il deploy dello scenario
- *Level 2*: un training di difficoltà media su topic come il network discovery, penetration testing e password cracking. Richiede due macchine virtuali per il deploy dello scenario.

Il processo di deploy del training, per una sola macchina virtuale, in CyTrONE avviene in 3 fasi distinte:

1. **Preparazione**: Su un nodo denominato *master* viene configurata (per esempio, accesso tramite SSH) ed avviata la macchina virtuale. T_p descrive il suo tempo di esecuzione
2. **Installazione**: Sulla macchina nel nodo master, vengono installati contenuti aggiuntivi necessari per il training, sulla macchina virtuale target. T_i descrive il suo tempo di esecuzione.
3. **Clonazione**: Copia tramite `parallel-scp` dell'immagine della macchina sui vari nodi della rete, garantendo al tempo stesso che password e credenziali siano distinte per ogni immagine. T_c descrive il suo tempo di esecuzione

   È dunque importante notare che per il *Level 1*, bisogna clonare con `parallel-scp` una sola immagine per nodo, mentre per il *Level 2* bisogna clonare 2 immagini per nodo.

Sia $T = T_p + T_i + T_c$, il grafico (Figura 2.4) descrive come si comporta T al variare del numero di host su cui fare il deploy delle macchine.

- Nei casi dove abbiamo solo 1 host, allora non c'è bisogno di clonare l'immagine, basta solo un host che esegua le 20 macchine virtuali, ossia il master. Dunque il tempo di provisioning è sotto i *due minuti*, mentre T per lo scenario *Level 2* si attesta sotto gli 8 minuti.

[10] https://www.sciencedirect.com/science/article/pii/S0167404818306527#bib0015

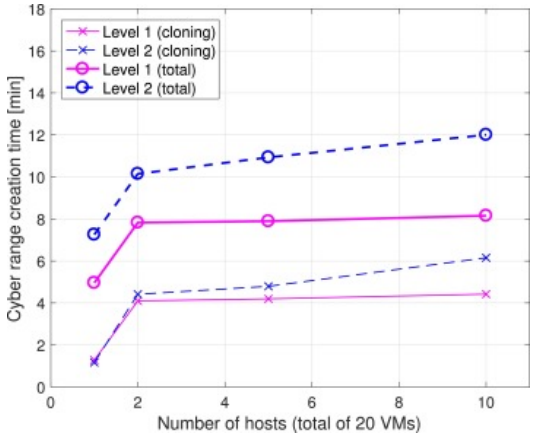


Figura 2.4: Valutazione dei tempi di provisioning in CyTrONE

- All'aumentare del numero n degli host, con il bisogno di clonare le macchine su più nodi, il tempo di provisioning totale tende a peggiorare, poiché bisogna trasferire l'immagine su più nodi, sebbene ogni nodo gestisca solo $20/n$ macchine virtuali.

Invece, per gli esperimenti su larga scala dove si vuole effettuare il deploy di 20 macchine per ogni host sui nodi disponibili, il tempo T cresce significativamente con il numero di istanze. Nella Figura 2.5 viene analizzato al variare del numero complessivo di macchine virtuali (fino a 600 istanze ripartite sui 30 nodi) cosa succede se ogni nodo deve eseguire 20 macchine, con 20 immagini differenti.

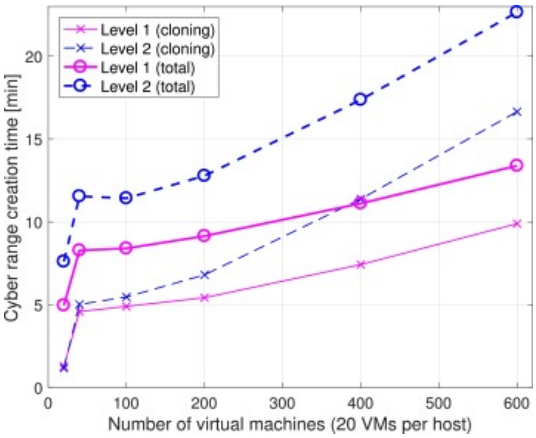


Figura 2.5: Scalabilità su larga scala del deployment in CyTrONE

Caratteristica	CyTrONE	BlueAgent
Virtualizzazione	Macchine Virtuali	Container di sistema
Infrastruttura	Cluster distribuito su larga scala (es: *StarBED*)	Singolo nodo host
Isolamento e Verifica	Il controller invia exploit via rete	Injection tramite Incus dei comandi all'interno di una rete inaccessibile al giocatore
Architettura di Rete	Reti virtuali KVM	Linux Bridge, SDN, WireGuard
Scopo Principale	Formazione didattica guidata con Moodle e verifiche statiche sulle macchine	Cyber Range compatto per competizioni di Difesa

2.4 CR Alternativi: CyberRangeCZ

CyberRangeCZ è un fork di **KYPO**. È una piattaforma basata su OpenStack per fornire un ambiente cloud su cui fare il deploy delle macchine. Utilizza tecnologie moderne come container, IaC (infrastructure as code) ed i microservizi.

Macchine virtuali, reti, e training sono definiti utilizzando JSON e YAML. Questi file vengono passati ad Ansible per descrivere i contenuti della macchina e a Packer per costruire le immagini delle macchine virtuali.

Fa uso massivo della virtualizzazione per ricreare fedelmente reti complesse ed interi sistemi operativi.

La piattaforma si concentra per soddisfare due casi d'uso principali:

- emulare intere infrastrutture di computer in ambienti controllati
- eseguire più sessioni di training contemporaneamente, gestendo centinaia di ambienti virtuali per centinaia o migliaia di partecipanti.

Il sistema si occupa di creare e approvvigionare un ambiente virtuale emulato costituito da Sandbox e Pool.

> **Sandbox**: *"È un ambiente di test isolato con reti virtuali che consente agli utenti di connettersi alle macchine virtuali (VM) e comunicare con il*

resto della rete, la macchina host e le altre VM. Sulle VM, l'utente può eseguire programmi, utilizzare servizi di rete o monitorare il traffico di rete, il tutto senza influire sull'infrastruttura esterna. La sandbox viene eseguita all'interno del cloud ed è accessibile da remoto." [11]

Pool: *"Un gruppo di sandbox cloud create a partire dalla medesima sandbox definition."* [11]

All'interno di una sandbox è possibile fornire una **Topology Definition**, un file che descrive la struttura dell'ambiente (host, router, reti, etc.) mentre il **Sandbox Provisioning** si occupa di customizzare le istanze nella topologia (Figura 2.6).

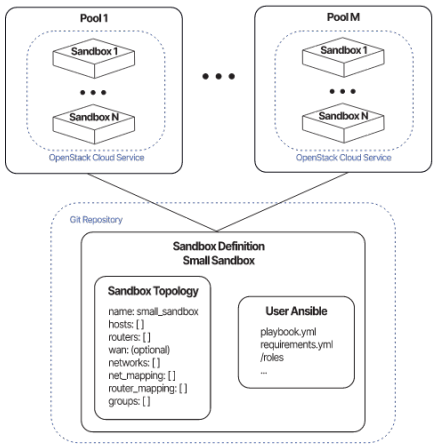


Figura 2.6: Elementi fondamentali e isolamento delle sandbox in KYPO

La creazione di una Sandbox si divide in 3 fasi:

- **Allocazione**: creazione della sandbox (=macchine virtuali) dentro al cloud.
- **Networking**: setup della rete e distribuzione delle chiavi di accesso dentro le macchine.
- **Provisioning**: customizzazione delle macchine virtuali nella sandbox.

La creazione della Pool precede necessariamente la creazione delle Sandbox.

A partire dalla Topology Definition, viene prodotta la **Topology Instance** utilizzando `Terraform` (che crea l'infrastruttura virtuale). L'accesso alla topologia è regolato

[11] https://docs.platform.cyberrange.cz/basic-concepts/terminology/#emulated-virtual-environment

da un MAN (Management Access Node), ossia l'unico nodo connesso all'esterno della rete della Topology Instance.

Di seguito, nella Figura 2.7, viene mostrata l'architettura generale di una Topology Instance.

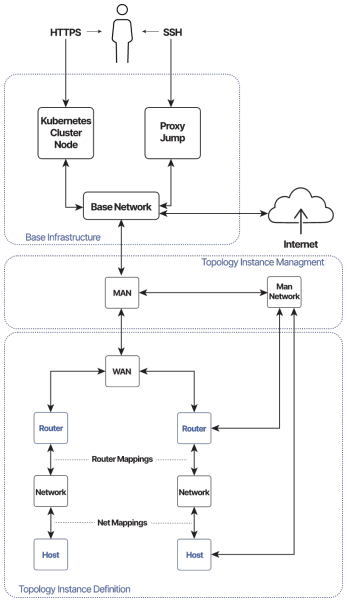


Figura 2.7: Istanziazione della topologia di rete in KYPO

L'accesso alle macchine dentro la Sandbox è regolato tramite SSH, oppure da una console web accessibile via browser.

Gestire Sandbox, ossia intere Topology Instance è costoso, infatti i requisiti minimi sono i seguenti:

- una macchina virtuale con almeno *2 core CPU, 16 GB di RAM e 40 GB di archiviazione*
- un server dotato di almeno *16 core CPU, 128 GB di RAM, 2 TB di archiviazione e almeno due interfacce di rete.*

Requisiti così alti sono necessari per:

- gestire l'infrastruttura OpenStack, i microservizi, e Kubernetes
- gestire l'esecuzione di centinaia di macchine virtuali e garantire la scalabilità al crescere degli utenti.

CyberRangeCZ a differenza di BlueAgent gestisce un'intera infrastruttura cloud, su cui vengono orchestrate macchine virtuali su OpenStack (configurate con Ansi-

ble) affiancate da microservizi gestiti tramite Kubernetes, mentre BlueAgent è un orchestratore di container che esegue su un unico host.

Caratteristica	CyberRangeCZ	BlueAgent
Virtualizzazione	Macchine Virtuali (VM)	Container di sistema (Incus)
Infrastruttura	Cluster Cloud distribuito (OpenStack, Kubernetes)	Singolo nodo host (Debian 13)
Requisiti Hardware	Elevati (16+ core, 128+ GB RAM, 2 TB disco)	Minimi (eseguibile su singolo server o PC economico)
Configurazione	Runtime con Ansible e template Packer	Creazione immagine con Distrobuilder
Rete	Rete virtuale OpenStack	Linux Bridge, WireGuard VPN e SDN
Verifica Difesa	Non prevista nativamente	Prevista
Scopo Principale	Emulazione reti aziendali e training su larga scala	Cyber Range compatto per CTF di difesa

2.5 CR Alternativi: CyberPatriot

È un programma di educazione nazionale per il K-12 creato dalla *Air & Space Forces Association* (AFA) [12]. I team assumono il ruolo di professionisti di cybersecurity che hanno l'obiettivo di proteggere vari sistemi entro un limite di tempo, analizzando le macchine virtuali, identificandone le vulnerabilità e mitigandole.

Ciascun team riceve una o più macchine virtuali complete pre-configurate (basate su distribuzioni Windows Server/Desktop o Linux), volutamente affette da molteplici difetti di sicurezza: account non autorizzati, policy di password permissive, servizi di rete non necessari in esecuzione, permessi errati sul filesystem e malware dormiente. I partecipanti operano come amministratori di sistema ("Blue Team"), con il compito di individuare e correggere tali falle entro una finestra temporale prefissata, mantenendo al contempo operativi i servizi critici per il business.

Il CyberPatriot Scoring System (CSS) tiene traccia della progressione del team e dei punteggi dei team. Un demone locale esegue controlli periodici sulla macchina per determinarne lo stato, mentre un server centrale registra i punteggi comunicando con i demoni delle macchine.

Questo sistema tuttavia è **closed-source** e solo l'**AFA** può utilizzarlo.

Caratteristica	CyberPatriot	BlueAgent
Virtualizzazione	Macchine Virtuali (VMware / VirtualBox)	Container di sistema (Incus)
Consumo di Risorse	Diversi GB di RAM per singola VM	~50 MB di RAM per container
Motore di Valutazione	Demone locale (CSS) all'interno della VM	Oracolo esterno in sandbox di rete isolata (OVN)

[12] Air & Space Forces Association, "CyberPatriot: The National Youth Cyber Education Program", https://www.uscyberpatriot.org

Capitolo 3

BlueAgent

Si presenta dunque l'architettura di *BlueAgent*, una piattaforma di Cyber Range in esecuzione come servizio *HTTP* che espone *API REST* per il deployment, la gestione del ciclo di vita e la verifica automatizzata delle challenge, a partire da definizioni dichiarative (*Infrastructure as Code*) predisposte dagli organizzatori della gara. Tali API REST possono essere consumate da un qualsiasi applicativo esterno denominato CTF Manager, come ad esempio un plugin per la piattaforma CTFd[1].

Il Cyber Range implementato da BlueAgent è pensato per scenari di Patch Management da eseguire su un singolo host. Si propone di offrire un ambiente basato su container di sistema, che garantisca un meccanismo di verifica completamente inaccessibile ai concorrenti. Tale meccanismo si fonda sulla clonazione all'occorrenza dell'intera infrastruttura del team in una rete logica separata, sulla quale vengono eseguiti i comandi di verifica definiti dall'oracolo.

I membri di ciascun team possono connettersi alla rete dei container a loro assegnata mediante una VPN WireGuard, dove a ciascun partecipante corrisponde un peer dedicato (ossia, coppia di chiavi pubblica e privata).

È opportuno evidenziare che:

1. Gli indirizzi IP dei container all'interno della rete di produzione di ciascun team sono statici e noti a priori (il che si applica alla maggior parte delle challenge).
2. La duplicazione istantanea sfrutta le proprietà Copy-on-Write (CoW) del filesystem ZFS. Inoltre viene impiegata una rete SDN (Software-Defined Network) per orchestrare router e switch logici dedicati all'ambiente di verifica.
3. BlueAgent non mantiene lo stato della competizione (quali i punteggi o la classifica dei team), delegando tale responsabilità al CTF Manager. BlueAgent opera

[1]CTFd LLC, *CTFd: Open Source Capture The Flag Framework*, https://ctfd.io/

esclusivamente come un orchestratore specializzato che traduce le specifiche delle challenge in container da eseguire.

A scopo d'esempio, mostriamo ad alto livello come si crea una challenge:

1. Il creatore della challenge definisce tramite un file YAML i container.
2. Il creatore della challenge, inserisce la nuova challenge tenendo in memoria tutte le informazioni necessarie a BlueAgent (inclusi i file dichiarativi)
3. Il giocatore di un determinato team accede alla challenge e tramite il CTF Manager richiede la creazione dei container.
4. BlueAgent si occupa di istanziare i container per il team.

BlueAgent è basato su *FastAPI*, per gestire in modo semplice ed efficiente le richieste HTTP.

L'host su cui viene eseguito BlueAgent deve mettere a disposizione dell'architettura le seguenti componenti:

- **Debian 13**: o una qualsiasi distribuzione Linux.
- `nftables`: per filtrare i pacchetti nel kernel
- **Python 3**: per eseguire BlueAgent.
- **Pool ZFS**: filesystem che supporta il meccanismo `COW`.
- **OVN ed Open vSwitch**: per gestire la rete di verifica dei container.
- **Distrobuilder**: per generare le immagini dei container in modo dichiarativo.
- **WireGuard**: per fornire una VPN come punto d'accesso ai partecipanti
- **Incus**: per la gestione dei container.

Per comprendere passo-passo cosa permette di fare il sistema e come lo fa sono elencati gli attori, gli use cases ed i requisiti di sistema.

3.1 Attori

L'attore è un'entità logica esterna al sistema che interagisce direttamente con esso, tramite l'invio di messaggi. È utile identificare ai fini dell'esposizione quali sono gli attori principali del sistema e che cosa si aspettano di poter fare.

3.1.1 CTF Manager (attore)

È un sistema software esterno a BlueAgent che si occupa di gestire la competizione (punteggi, team, utenti, configurazioni, gara). Comunica con BlueAgent tramite API

REST per gestire l'intero ciclo di vita delle macchine dei team, ed agisce come entità autenticata (tramite l'utilizzo di un token) per BlueAgent. Mantiene lo stato delle challenge, siccome BlueAgent, per design non lo fa.

Richiede per conto dei Giocatori il deploy delle challenge, la verifica dei container e le configurazioni per accedere alla rete tramite VPN.

Per conto dell'autore della sfida permette la gestione dei file di configurazione delle challenge. Permette invece all'admin di gestire i container attivi, i team, e altri aspetti della gara.

Più avanti, verrà presentato un plugin per CTFd, che agisce come CTF Manager verso BlueAgent.

Dunque CTF Manager è l'unico attore per BlueAgent e rendere il CTF Manager esterno ed indipendente da BlueAgent nella sua architettura, permette di non vincolare l'uso di BlueAgent con CTFd.

È opportuno definire il ruolo dell'Amministratore, del Giocatore e dell'Autore della sfida, evidenziando come tali figure non interagiscano direttamente con BlueAgent bensì mediatamente tramite il CTF Manager.

3.1.2 Giocatore (attore esterno per CTF Manager)

È un'entità che agisce in team con altri giocatori, oppure da solo. Ha accesso alle macchine delle challenge istanziate da BlueAgent, appartenenti al suo team. Non comunica direttamente con BlueAgent ma piuttosto si interfaccia con il CTF Manager per gestire la progressione nella competizione e per richiedere il deploy delle macchine.

3.1.3 Autore della sfida (attore esterno per CTF Manager)

Si occupa di creare i file di configurazione per una o più challenge e di inviare tramite CTF Manager le immagini dei container definite tramite configurazione distrobuilder. Dunque non dialoga direttamente con BlueAgent.

3.1.4 Amministratore della competizione (attore esterno per CTF Manager)

Per aspetti legati alla gestione della competizione CTF, l'amministratore si interfaccia direttamente con il CTF Manager. Se ha bisogno di interagire con configurazioni di team e container, il CTF Manager cattura le richieste dell'amministratore (tramite interfaccia apposita) e le inoltra a BlueAgent.

3.2 Use Cases per CTF Manager

Sono elencati di seguito gli Use Case per il CTF Manager, arricchito con alcuni dettagli implementativi per comprendere al meglio l'architettura del software.

Di seguito è riportato un breve glossario:

`br-team-k`: è il bridge di rete che fornisce una rete locale per i container del team k. Ogni `br-team-k` è una LAN separata dagli altri `br-team-k`.

`wg-k`: è l'interfaccia WireGuard che offre la connettività VPN per il team k verso `br-team-k`.

`eth0`: in questo documento, spesso identifica il nome dell'interfaccia che fornisce la connettività alla LAN per un container.

`verify-k`: è lo switch logico (gestito tramite SDN) utilizzato per implementare la rete di verifica per il team k. Questa rete è completamente scollegata da `br-team-k` e da `wg-k`.

`database`: è il database SQL, usato da BlueAgent per tenere traccia delle risorse allocate.

3.2.1 `UC-01`: Deploy dell'infrastruttura di rete per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Effettuare il deploy delle interfacce di rete necessarie al team k per eseguire i container e consentire l'accesso VPN ai partecipanti.
- **Precondizione**: BlueAgent è attivo ed è raggiungibile via API REST.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `PUT /teams/{team_k}`, specificando l'identificativo k nel path ed il relativo nome nel payload.
2. BlueAgent verifica se l'infrastruttura per il team k esiste già:
    1. Se il team è già registrato, ne sincronizza la configurazione e risponde `HTTP 200 OK`.
    2. Altrimenti, l'infrastruttura di rete viene creata da zero:
        1. Viene creato il bridge `br-team-k` per il team k con IP per il gateway `10.k.1.1/24`.

2. Viene creata l'interfaccia WireGuard `wg-k` per il team k. La porta UDP è `51820+k`, l'IP per il gateway è `10.k.0.1/24`.
3. Viene registrato il team all'interno del database.
4. BlueAgent risponde con stato `HTTP 201 Created` confermando l'avvenuta creazione.

Se avviene un errore irrimediabile durante la creazione, il bridge e l'interfaccia WireGuard create vengono rilasciate.

3.2.2 `UC-02`: Deploy della challenge per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Effettuare il deploy dei container necessari alla challenge da allestire per il team k richiedente.
- **Precondizione**: `UC-01`, `UC-05`. È stato eseguito con successo il deploy dell'infrastruttura di rete per il team k e le immagini dei container richieste sono presenti nel catalogo in stato `ready`.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/machines` specificando la configurazione della challenge.
2. BlueAgent valida le precondizioni.
3. Per ogni container specificato nella configurazione:
    1. Crea l'istanza del container.
    2. Collega l'interfaccia virtuale `eth0` dell'istanza al bridge del team `br-team-k`.
    3. Inietta nel filesystem dell'istanza la configurazione di rete statica.
    4. Avvia l'istanza del container ed applica l'indirizzo IP statico.
    5. Registra il container nel database.
4. BlueAgent restituisce la risposta con stato `HTTP 201 Created`.

Se si verifica un errore durante l'avvio, BlueAgent arresta ed elimina tutte le istanze e le risorse create fino a quel momento per la challenge e risponde con un errore.

3.2.3 `UC-03`: Verifica della challenge per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Verificare l'integrità dei servizi e l'efficacia delle patch applicate dal team mediante l'esecuzione dei comandi specificati nella configurazione della challenge.
- **Precondizioni**: `UC-02`. I container relativi alla challenge specificata per il team k sono attivi.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/verificator` trasmettendo la configurazione della challenge da verificare.
2. Se una verifica per lo stesso team è già in corso, BlueAgent risponde con `HTTP 409 Conflict`.
3. Altrimenti, BlueAgent predispone l'ambiente di verifica:
    1. Crea uno switch logico OVN denominato `verify-k`.
    2. Esegue la clonazione CoW di tutti i container della challenge del team k.
    3. Riconfigura l'interfaccia `eth0` dei cloni agganciandola allo switch OVN di verifica.
    4. Avvia i cloni sandbox e configura al loro interno lo stesso indirizzo IP statico di produzione (`10.k.1.{ip_suffix}/24`).
    5. Se specificato nella configurazione, avvia un container di verifica runner.
4. Esecuzione dei controlli di readiness: viene eseguito un ciclo di tentativi finché il comando non ha successo, oppure fino allo scadere del timeout.
5. Esegue i comandi di verifica all'interno dei container clonati o dal container runner.
6. Dealloca le risorse sandbox create per il controllo.
7. BlueAgent risponde con stato `HTTP 200 OK` comunicando l'esito della verifica.

Se si verifica un errore, oppure i comandi di readiness non hanno successo, allora le risorse utilizzate vengono rilasciate e viene comunicato l'errore.

3.2.4 `UC-04`: Dismissione e rilascio di una challenge per il team k

- **Attore**: CTF Manager

- **Obiettivo**: Rimuovere tutti i container associati a una determinata challenge per il team k.
- **Precondizioni**: I container associati alla challenge specificata sono attivi per il team k.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /teams/{team_k}/machines` specificando nel payload la configurazione della challenge.
2. Per ciascun container specificato nella configurazione della challenge:
    1. Arresta l'istanza.
    2. Elimina l'istanza.
    3. Cancella il record del container dal database.
3. BlueAgent risponde con stato `HTTP 204 No Content`.

In caso di problemi durante il teardown di uno o più container, BlueAgent restituisce un errore HTTP con i dettagli dell'anomalia.

3.2.5 `UC-05`: Compilazione e Import di un'immagine per container

- **Attore**: CTF Manager
- **Obiettivo**: Inviare la specifica di un'immagine in formato YAML ed avviare la compilazione e l'importazione nel catalogo delle immagini.
- **Precondizione**: BlueAgent è attivo.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `PUT /images/{alias}` inviando una form multipart con il file di definizione YAML o l'archivio di contesto.
2. BlueAgent valida la specifica YAML e ne calcola l'hash crittografico.
3. BlueAgent verifica lo stato dell'immagine nel database:
    1. Se l'immagine con lo stesso hash esiste già ed è in stato `ready`, risponde immediatamente con `HTTP 200 OK`.
    2. Altrimenti, registra un nuovo record nel database in stato `building`.
4. BlueAgent avvia la compilazione tramite un worker asincrono:

1. Esegue Distrobuilder ed importa l'immagine generata nel catalogo dei container Incus.
2. Assegna l'alias all'immagine importata e aggiorna il record a `ready` nel database.

5. BlueAgent restituisce immediatamente al CTF Manager lo stato `HTTP 202 Accepted` se la compilazione è stata avviata.

3.2.6 `UC-06`: Registrazione e configurazione WireGuard per un membro del team k

- **Attore**: CTF Manager
- **Obiettivo**: Registrare un giocatore come membro del team k, allocare un indirizzo IP nella subnet VPN, generare le chiavi crittografiche WireGuard e restituire il file di configurazione `.conf`.
- **Precondizioni**: `UC-01`. L'infrastruttura di rete per il team k è attiva sul sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `PUT /teams/{team_k}/members/{user_id}` fornendo il nome dell'utente ed opzionalmente la chiave pubblica WireGuard.
2. BlueAgent verifica se il team k è registrato; se assente, risponde con `HTTP 404 Not Found`.
3. BlueAgent verifica se il membro è già registrato nel database per il team k:
    1. Se il membro esiste già, BlueAgent aggiorna la configurazione rispondendo con `HTTP 200 OK`.
    2. Se il membro è nuovo:
        1. Alloca un indirizzo IP VPN libero per il team (`10.k.0.2` - `10.k.0.254`). Se lo spazio degli indirizzi è esaurito, risponde con `HTTP 409 Conflict`.
        2. Se non fornita, genera una nuova coppia di chiavi asimmetriche.
        3. Registra il nuovo peer sull'interfaccia `wg-k`.
        4. Salva il record del membro nel database.
4. BlueAgent risponde con stato `HTTP 201 Created` restituendo i dati del membro e la configurazione WireGuard generata.

3.3 Requisiti di Sistema

I requisiti di sistema definiscono i vincoli architetturali, infrastrutturali e operativi che BlueAgent deve garantire per supportare i casi d'uso descritti. Di seguito vengono presentati, suddivisi nelle rispettive categorie funzionali, i requisiti formali di progetto.

3.3.1 Piattaforma Host e Infrastruttura

La tabella seguente riassume i requisiti relativi al sistema operativo host e ai componenti infrastrutturali di base necessari all'esecuzione di BlueAgent.

Codice	Requisito	Descrizione Operativa
SR-HST-01	Sistema Operativo Host	L'host deve eseguire un kernel Linux, con IP forwarding abilitato e supporto completo per il sottosistema `nftables`.
SR-HST-02	Storage Pool ZFS	L'host deve disporre di uno storage pool ZFS (`blueagent-zfs`) con supporto nativo a snapshot Copy-on-Write (CoW).
SR-HST-03	Servizi SDN Host	L'host deve eseguire i demoni Open vSwitch e OVN (`ovs-vswitchd`, `ovn-northd`, `ovn-controller`) per il provisioning dinamico degli switch logici effimeri.
SR-HST-04	Supporto WireGuard	L'host deve fornire il modulo kernel WireGuard e i relativi strumenti di configurazione.
SR-HST-05	Pipeline Distrobuilder	L'host deve disporre di `distrobuilder` per la compilazione automatizzata e riproducibile delle immagini dei container.

3.3.2 Motore di Containerizzazione

I requisiti per il motore di containerizzazione stabiliscono i meccanismi di interazione con Incus, l'isolamento dei carichi e la gestione del ciclo di vita dei container di sistema.

Codice	Requisito	Descrizione Operativa
SR-ENG-01	Astrazione del Motore	Disaccoppiamento della logica di orchestrazione tramite l'interfaccia astratta `ContainerEngine`.
SR-ENG-02	Client Asincrono UDS	Comunicazione non bloccante con Incus via Unix Domain Socket (`/var/lib/incus/unix.socket`) mediante client HTTP asincrono.
SR-ENG-03	Limitazione Risorse (Cgroups v2)	Applicazione di quote hardware per impedire la starvation delle risorse dell'host.
SR-ENG-04	Clonazione Snapshot CoW	Duplicazione istantanea delle istanze container basata su snapshot CoW a livello di storage pool ZFS.

3.3.3 Rete Ibrida e Connettività

Questa sezione elenca i requisiti di rete necessari a garantire prestazioni native in fase di gara e isolamento crittografico per ogni squadra.

Codice	Requisito	Descrizione Operativa
SR-NET-01	Gateway VPN WireGuard	Allocazione di un'interfaccia kernel `wg-k` per ciascun team k, in ascolto sulla porta UDP $51820 + k$ con indirizzo gateway `10.k.0.1/24`.
SR-NET-02	Linux Bridge L2 Live	Allocazione di un bridge Linux `br-team-k` per ciascun team k, con gateway `10.k.1.1/24`.
SR-NET-03	Indirizzamento e Credenziali VPN	Allocazione deterministica degli IP dei membri nella subnet `10.k.0.0/24`.
SR-NET-04	Topologia Deterministica	Configurazione statica degli indirizzi IP `10.k.1.x` nei container in base al suffisso (`ip_suffix`) dichiarato nella specifica della challenge.
SR-NET-05	TCP MSS Clamping	Applicazione di regole `nftables` per forzare il clamping del MSS, prevenendo pacchetti persi dovuti al tunnel WireGuard.

Codice	Requisito	Descrizione Operativa
SR-NET-06	Isolamento e Masquerading NAT	Isolamento inter-team tramite il set `@team_links` e masquerading del traffico outbound verso la rete esterna per l'accesso a Internet per giocatori, container e rete di verifica.

3.3.4 Motore di Verifica OVN Sandbox

I requisiti del motore di verifica definiscono le garanzie di non interferenza, trasparenza e sicurezza durante l'esecuzione degli exploit e dei test di integrità.

Codice	Requisito	Descrizione Operativa
SR-VRF-01	Switch Logico OVN	Provisioning di uno switch virtuale OVN (`verify-<session>`) senza esporre il gateway all'host per evitare collisioni di rotte tra container.
SR-VRF-02	Fedeltà Topologica	I cloni nella sandbox preservano l'interfaccia `eth0` e la medesima configurazione IP (`10.k.1.x`) specificata.
SR-VRF-03	Readiness Probes e Timeout	Esecuzione preliminare dei controlli di prontezza sui servizi con tentativi periodici, prima dell'esecuzione dei check di verifica.
SR-VRF-04	Confinamento Zero Side-Channel	Isolamento del traffico di verifica sullo switch OVN per impedire qualsiasi sniffing (es. `tcpdump`) da parte dei concorrenti.
SR-VRF-05	Rilascio Automatico Risorse	Deallocazione automatica e atomica di cloni, switch e porte virtuali al termine della verifica o su eccezione.

3.3.5 Control Plane, Concorrenza e Persistenza

La tabella seguente specifica i vincoli di sincronizzazione, resilienza e coerenza dello stato applicativo gestiti dal control plane di BlueAgent.

Codice	Requisito	Descrizione Operativa
SR-CON-01	Two-Tier Lock Registry	Meccanismo di lock gerarchico a due livelli per prevenire race condition sulle risorse.
SR-CON-02	Self-Healing e Riconciliazione	Demone `LifecycleManager` per la bonifica automatica degli switch e cloni orfani.
SR-CON-03	Healthcheck & Telemetria Live	Endpoint `/health` per ottenere lo stato di salute del sistema. Raccolta telemetrica dello stato dei peer WireGuard.
SR-CON-04	Persistenza Asincrona SQLite	Memorizzazione dello stato di team, membri, container e cataloghi su database SQLite.
SR-CON-05	Rollback Atomico su Errore	Qualsiasi fallimento durante il provisioning di container o interfacce di rete innesca la deallocazione delle risorse.
SR-CON-06	Idempotenza Operativa	Tutte le richieste di creazione e configurazione per risorse già esistenti non generano errori applicativi.
SR-CON-07	Autenticazione Centralizzata	Validazione obbligatoria dell'accesso a tutti gli endpoint tramite token segreto condiviso.

3.3.6 Pipeline Immagini Distrobuilder

I requisiti della pipeline stabiliscono la creazione riproducibile delle immagini per le sfide, il loro tracciamento e il caching dei binari.

Codice	Requisito	Descrizione Operativa
SR-IMG-01	Definizione IaC Distrobuilder	Creazione dichiarativa di immagini per container di sistema a partire da manifesti YAML versionabili.
SR-IMG-02	Caching e Identificazione SHA-256	Tracciamento crittografico della specifica YAML tramite hash SHA-256 per identificare modifiche su immagini già compilate nel database.

Codice	Requisito	Descrizione Operativa
`SR-IMG-03`	Compilazione Asincrona	Esecuzione delle fasi di build e import Incus in background tramite task asincrono con tracciamento dei log.

3.4 Meccanismi di Verifica ed Oracoli

Per decidere se un team ha recuperato la **flag**, bisogna definire un ***oracolo***, che determini lo stato in cui si trova il sistema target.

L'oracolo è in grado di accedere al sistema e di compiere due azioni: può leggere il filesystem di qualsiasi nodo e può eseguire comandi su qualsiasi nodo.

Per ogni tipo di azione eseguita, l'oracolo determina lo stato del sistema confrontando il risultato dell'azione con il valore atteso (validazione degli output). In BlueAgent, l'oracolo è l'entità che esegue sui container clonati i comandi specificati dall'Autore della sfida, ossia nel codice la classe `OVNVerifier`.

Dunque all'interno del processo di verifica, il sistema si può definire *patchato* se e soltanto se tutte le azioni sono validate rispetto al valore atteso in output.

Di seguito sono discussi alcuni meccanismi di verifica che abbiamo identificato, al fine di motivare la clonazione dei container implementata in BlueAgent.

3.4.1 Verifica Statica

La verifica statica prevede che l'oracolo possa accedere al filesystem, comparando le patch applicate nei file con una patch considerata di riferimento per l'oracolo.

Questo tipo di verifica può essere implementato in vari modi:

- **diff**: ad ogni file interessato corrisponde un file di riferimento per l'oracolo. Ogni file viene dunque confrontato, riga per riga, con il suo riferimento per determinare se il team ha applicato la patch correttamente.
- **SAST**: Static Analysis Security Testing. Prevede l'accesso al codice sorgente non compilato in modo da analizzare pattern di codice vulnerabili e fallacie logiche.[2]

[2]S. S. A. A. S. et al., *Current Status of Static Application Security Testing (SAST)*, Annals of Emerging Technologies in Computing (AETiC), Vol. 4, No. 3, 2020. https://aetic.theiaer.org/archive/v4/v4n3/p1.html

Tuttavia, SAST non è un metodo di analisi efficace, sebbene efficiente:

- Possono essere identificati nel processo di verifica, vari falsi positivi (e falsi negativi)[2]
- Non è possibile identificare staticamente tutti i tipi di errori di sicurezza [2]
- Non è possibile identificare vulnerabilità all'interno dei binari compilati[2]

Dunque l'analisi statica può in alcuni casi risultare inesatta, e non coprire tutti i casi d'uso per quanto riguarda la gestione di competizioni CTF.

3.4.2 Verifica Dinamica tramite orchestratore esterno

Questo meccanismo di verifica prevede l'uso di un **orchestratore esterno** alle macchine vulnerabili, ossia l'attaccante, che può interagire con la vulnerabilità utilizzando la rete locale. L'attaccante orchestra l'esecuzione di uno o più comandi, con l'obiettivo di inviare il payload all'interno di un pacchetto di rete.

Durante una competizione di questo tipo, tuttavia, i membri dei team sono legittimati ad accedere alla macchina vulnerabile come utente `root`: così da permettere l'applicazione delle patch sui file di sistema. Questo implica che i partecipanti dei team siano in grado di eseguire tool come `tcpdump` e `wireshark` per intercettare il traffico di rete e di conseguenza, di visualizzarne il payload, ottenendo un suggerimento per la soluzione della challenge.

Inoltre, il payload inviato potrebbe essere tale da mandare in crash il servizio target, causando una penalità alla percentuale di **SLA** *(Service Level Agreement)*, ed interferendo potenzialmente con il lavoro del team sfidante (portando per esempio alla perdita di alcuni file). Si vuole dunque trovare un sistema di verifica che esegua completamente all'oscuro del team sfidante.

Il sistema di verifica tramite sistema esterno non copre nemmeno tutte le vulnerabilità. Banalmente, non tutti i servizi vulnerabili sono esposti in rete, ma sono accessibili solo in locale.

Riassumendo quindi, abbiamo identificato alcuni spunti per ideare un meccanismo di verifica migliore:

- Il traffico di rete non dovrebbe essere letto dal team sfidante al fine di ottenere suggerimenti per la soluzione.
- Se l'exploit manda in crash il servizio, questo non dovrebbe penalizzare il team, o più in generale: il team sfidante non vede il servizio distruggersi.
- Dobbiamo gestire anche vulnerabilità non esposte in rete.

3.4.3 Verifica Dinamica tramite injection dei comandi di verifica

Questo meccanismo di verifica prevede l'uso di un **orchestratore esterno** al sistema vulnerabile, che ha la facoltà di eseguire comandi all'interno delle macchine vulnerabili che compongono il sistema. Dunque l'orchestratore esegue uno o più comandi con il fine di verificare che il sistema sia resistente alla vulnerabilità.

Considerando il caso in cui una macchina `A` contenga un file binario vulnerabile ad un *buffer overflow*, l'orchestratore potrebbe eseguire ad esempio il comando `/path/to/service_logger payload` e verificare l'esito dell'esecuzione.

L'esecuzione di questi comandi è possibile per esempio in ambienti containerizzati, sfruttando utility come `docker exec` o `incus exec`, che permettono di iniettare comandi all'interno dei container.

Tuttavia: comandi come `pspy`, (che ascoltano attivamente le chiamate di sistema effettuate e gli eventi sul file system in `/proc`) possono intercettare l'esecuzione dei comandi dell'orchestratore e di mostrarli al team, suggerendo al team sfidante la soluzione. [3]

Dunque, dobbiamo ideare un meccanismo di verifica che sia completamente invisibile a questi meccanismi di intercettazione.

3.4.4 Clonazione del sistema in una rete separata

Clonare i container all'interno di una rete separata ed inaccessibile ai giocatori è una soluzione plausibile in quanto:

- Isola la rete rispetto alla rete dei giocatori. I giocatori non possono intercettare il traffico di rete.
- I giocatori, per design, non hanno accesso ai container duplicati dunque non possono eseguire alcun tipo di comando.

Tuttavia introduce un costo in termini di storage e di utilizzo della RAM dovuto alla duplicazione dei container, anche se questi esistono solo per il tempo necessario all'esecuzione dei comandi definiti dal creatore della sfida. Questo aspetto verrà misurato più avanti e sarà oggetto di analisi.

Altri meccanismi di verifica ed architetture di rete possibili sono analizzati nel Capitolo 6.

[3] D. Breuker, *pspy: unprivileged Linux process snooping*, https://github.com/DominicBreuker/pspy

Capitolo 4

Architettura di Rete ad Alto Livello

La topologia di rete di Blue Agent è suddivisa logicamente in due reti:

- **Una rete di produzione per ogni team**, dove la rete production-k identifica gli oggetti di rete ed i container necessari al team k.
- **Al più una rete di verifica per ogni team**, dove la rete verify-k identifica gli oggetti di rete ed i container duplicati utilizzati per la verifica richiesta dal team k.

Lo schema di indirizzamento prevede una netta separazione tra lo spazio di gioco del team k, la rete WireGuard e la rete di Incus:

- `10.k.0.0/24`: è la sottorete WireGuard per i membri del team k.
- `10.k.1.0/24`: è la sottorete LAN per i container di produzione e per i cloni di verifica.
- `172.16.0.0/24`: è la sottorete gestita da incusbr0 per fornire la connettività esterna e le reti OVN tramite NAT.

4.1 Rete production-k

La rete production-k (Figura 4.1) è progettata per offrire un ambiente isolato e raggiungibile esclusivamente dai membri autorizzati del team. È costituita dai seguenti dispositivi di rete:

- wg-k: dispositivo L3 gestito da WireGuard, fa da terminazione per il tunnel del team k-esimo. Ha come IP gateway: `10.k.0.1/24`
- br-team-k: dispositivo L2 gestito dal kernel Linux. Ha come IP gateway: `10.k.1.1/24`

- eth0: le interfacce di rete eth0 dei container che vengono collegate a br-team-k.

wg-k serve per fornire ai membri del team k l'endpoint per la connessione tramite VPN, in questo modo i membri del team saranno connessi direttamente alla rete production-k. Ogni membro ha la sua configurazione WireGuard associata al suo peer impostato sull'interfaccia wg-k. Aggiungere un peer all'interfaccia WireGuard significa permettere ad un client con una certa chiave pubblica di collegarsi con un certo indirizzo ip (**cryptokey routing**), e quindi assegnare ad ogni membro del team k un indirizzo nella sottorete `10.k.0.0/24`.

L'interfaccia br-team-k permette di creare una rete LAN per i container del team k, a cui questi si collegano.

Questo collegamento avviene tramite l'utilizzo di una `veth pair` (Virtual Ethernet) per container: un'estremità è collegata a br-team-k, l'altra all'interfaccia del container `eth0`. Questa architettura comporta la creazione di un numero di `veth` pair lineare al numero di container attivi.

> `veth`: È utile per collegare interfacce che risiedono per esempio su differenti Network Namespace (ossia il modo in cui sono implementati i container). Utilizzano poca CPU e poca memoria, dato che l'implementazione è efficiente e risiede direttamente nel kernel. Dunque per centinaia di `veth` non si ha un degrado evidente delle prestazioni. [1]

Tramite regole di forwarding del kernel e di firewall via `nftables`, il traffico dell'interfaccia wg-k viene autorizzato verso br-team-k e viceversa. Questa regola viene applicata aggiungendo dentro al set `team_links` (una struttura dati all'interno del kernel gestita da nftables) la coppia di interfacce interessate.

4.2 Rete verify-k

La rete verify-k (Figura 4.2) è costituita dai seguenti componenti:

- verify-k: rete virtuale logica gestita via SDN tramite OVN e orchestrata da Incus [2]. Non costituisce una singola interfaccia di rete convenzionale del kernel, ma una topologia virtuale isolata composta da uno switch e un router con gateway: `10.k.1.1/24`.

[1] Linux manual page, veth(4) - Virtual Ethernet Device: https://man7.org/linux/man-pages/man4/veth.4.html

[2] Documentazione ufficiale Incus, OVN network driver: https://linuxcontainers.org/incus/docs/main/reference/

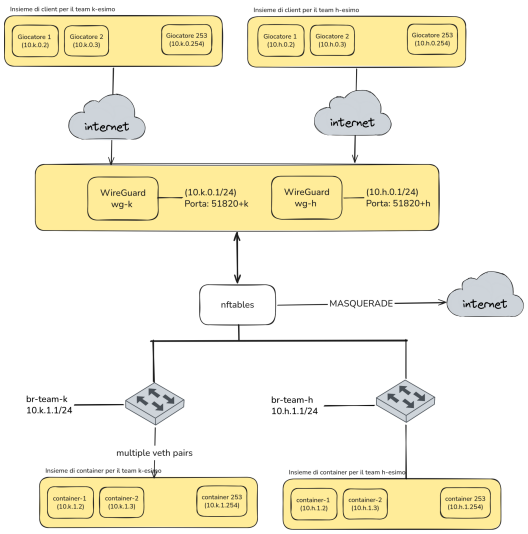


Figura 4.1: Topologia della rete di produzione (production-k) per il team k

- incusbr0: bridge di rete di Incus con gateway `172.16.0.1/24`. Svolge il ruolo di interfaccia per l'instradamento verso l'esterno della rete OVN.
- eth0: sono le interfacce di rete dei container clonati (`10.k.1.x`) che vengono collegate a verify-k.

  Incus è in grado di gestire da solo il collegamento tra incusbr0 e verify-k. Questo semplifica di molto la gestione delle interfacce di verifica per BlueAgent.

Le interfacce dei container e la rete verify-k sono gestite dalla SDN (*Software Defined Network*). verify-k è implementata da:

1. un **Logical Switch** (denominato internamente da Incus nel DB di OVN come `incus-net<ID>-ls-int` [2]): rappresenta la LAN su cui si agganciano le interfacce virtuali dei container di verifica.
2. un **Logical Router** (denominato `incus-net<ID>-lr` [2]): agisce come gateway (`10.k.1.1/24`), permettendo ai container clonati di preservare l'identica configurazione IP rispetto alla produzione.

   Tunneling Geneve e considerazioni sull'MTU: OVN impiega il protocollo di tunneling **Geneve** (*Generic Network Virtualization Encapsulation*) [3] per spostare i pacchetti tra più nodi della rete.

[3] Documentazione architetturale OVN e Geneve: https://docs.ovn.org/en/latest/topics/geneve.html

L'incapsulamento Geneve introduce tuttavia un overhead di intestazione variabile (fino a 58 byte), riducendo l'MTU effettiva della rete a 1442 byte (rispetto ai 1500 byte dello standard Ethernet). Se pacchetti di dimensione eccedente hanno il flag *Don't Fragment* (DF) abilitato, rischiano di essere scartati generando un fenomeno di *Path MTU Blackhole*.

Bisogna notare che il tunnel Geneve non viene mai utilizzato, poiché i container di verifica risiedono tutti all'interno dello stesso nodo. Tuttavia Incus limita di default l'MTU delle interfacce di rete dei container a 1442 byte per precauzione; per risolvere il problema è possibile forzare l'MTU a 1500.

L'uso di una SDN garantisce l'assenza di conflitti di routing grazie a una netta separazione tra **spazio utente** e **spazio kernel** [4]:

- **Spazio Utente (Control Plane):** i demoni di controllo di OVN (`ovn-northd` e l'agente locale `ovn-controller`) elaborano la topologia logica e la traducono in flussi OpenFlow concreti per Open vSwitch (`ovs-vswitchd`). Il router OVN esiste puramente a livello logico.
- **Spazio Kernel (Data Plane):** il demone `ovs-vswitchd` riceve i flussi OpenFlow e programma il modulo kernel `openvswitch.ko`, dove viene applicato l'inoltro dei frame ad alte prestazioni.

Il collegamento tra i container e la rete OVN sfrutta sempre una coppia `veth`: un'estremità risiede all'interno del container e si attacca ad `eth0`, mentre l'altra estremità sull'host viene agganciata direttamente al bridge di Open vSwitch.

> `SDN (Software Defined Networking)`: Paradigma architetturale che separa il **piano di controllo** dal **piano dei dati**, consentendo di gestire e programmare l'infrastruttura di rete interamente via software.
>
> `OpenFlow`: Protocollo standard che permette al controller SDN di comunicare con lo switch e programmare le regole di matching e le azioni direttamente all'interno delle pipeline di Open vSwitch.
>
> `OVN (Open Virtual Network)`: Sistema SDN che opera come Control Plane per Open vSwitch. Traduce topologie logiche (Logical Switch, Logical Router, NAT e ACL) tramite `ovn-northd` e `ovn-controller` in flussi OpenFlow distribuiti, istruendo Open vSwitch su come instradare il traffico senza creare

[4] Documentazione Open vSwitch, architettura Datapath Kernel vs Userspace: https://docs.openvswitch.org/en/latest/intro/what-is-ovs/

molteplici interfacce o bridge dedicati nel kernel host. È costituito da due database: **OVN Northbound**: memorizza la dichiarazione delle reti logiche da creare e le proprietà che devono rispettare. **OVN Southbound:** memorizza le tabelle di flussi logici compilate da `ovn-northd`, che `ovn-controller` traduce in regole OpenFlow concrete per il piano dei dati.

La comunicazione verso l'esterno avviene collegando il Logical Router OVN al bridge parent incusbr0, da cui ottiene un indirizzo IP da utilizzare. L'accesso a Internet per i container di verifica sfrutta un meccanismo di NAT a due stadi:

- **Primo Stadio (SNAT in OVN)**: quando un container di verifica (`10.k.1.x`) invia traffico all'esterno, il Logical Router riscrive l'indirizzo sorgente sostituendolo con l'IP assegnatogli da `incusbr0` (nella sottorete `172.16.0.0/24`). Se non fosse applicato, i pacchetti in uscita manterrebbero l'indirizzo sorgente `10.k.1.x`, i quali sono attestati sulle interfacce di rete dei container nella rete di produzione.
- **Secondo Stadio (SNAT su `incusbr0`)**: una volta raggiunto `incusbr0`, il sottosistema Netfilter, tramite regole `nftables`, applica la regola di **masquerade** verso l'interfaccia fisica per uscire su Internet. Viene sostituito l'indirizzo di `incusbr0` con l'indirizzo pubblico dell'host che esegue BlueAgent.

Tratta del Pacchetto	Indirizzo Sorgente (IP_{src})	Indirizzo Destinazione (IP_{dst})	Responsabile
Dal Container ad `incusbr0`	`10.k.1.x` *(IP duplicato)*	IP_{ext} *(Internet)*	Container di verifica
Da `incusbr0` all'interfaccia dell'host	`172.16.0.x` *(IP transito univoco)*	IP_{ext} *(Internet)*	Incus
Dall'host verso l'esterno	IP dell'interfaccia pubblica dell'host.	IP_{ext} *(Internet)*	`nftables` / Netfilter

Il NAT a due stadi è necessario per evitare collisioni di routing sull'Host: il primo stadio "nasconde" gli indirizzi duplicati `10.k.1.x` traducendoli in un IP su `172.16.0.0/24`, mentre il secondo stadio permette la navigazione esterna condividendo l'IP dell'host.

`NAT (Network Address Translation)`: Meccanismo di rete che modifica le informazioni di intestazione IP (e opzionalmente le porte di livello 4) dei pacchetti in fase di routing. [5].

[5]Documentazione Netfilter, Network Address Translation (NAT): https://wiki.nftables.org/wiki-nftables/index.php/Performing_Network_Address_Translation_(NAT)

`SNAT (Source NAT)`: Variante del NAT applicata al traffico uscente (**egress**, fase di postrouting). Modifica l'indirizzo IP sorgente del pacchetto con un indirizzo pubblico o instradabile dalla rete esterna.

`Masquerading`: Variante dinamica di `SNAT`. Si ha quando l'indirizzo sorgente viene mappato dinamicamente sull'IP dell'interfaccia di uscita dell'host.

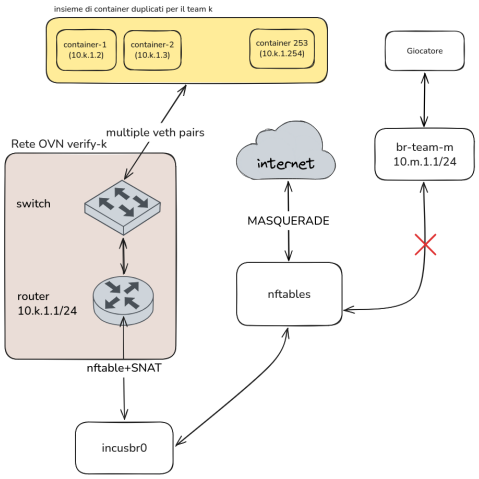


Figura 4.2: Topologia della rete di verifica sandbox (verify-k) orchestrata con OVN

4.3 Struttura e configurazione di una Challenge

In BlueAgent, la definizione di una challenge adotta l'approccio *Infrastructure-as-Code* (IaC): l'intera topologia viene descritta mediante uno o più file dichiarativi in formato YAML.

Questo file specifica quali container compongono l'ambiente, le risorse hardware assegnate a ciascuno di essi ed i comandi di readiness e di verifica necessari per convalidarne la patch.

La configurazione della challenge avviene in due step:

- Specifica delle immagini Distrobuilder tramite file di configurazione.
- Configurazione della challenge vera e propria. Vengono specificati qui i container da istanziare e con quali immagini.

4.3.1 Struttura del File di Configurazione della Challenge

La struttura della specifica di una challenge è articolata nelle seguenti sezioni principali:

1. **Metadati e Identificazione:**
   - `challenge_id`: Identificatore alfanumerico univoco globale della sfida (ad esempio `patch-web-sqli`).
   - `team_k`: Indice numerico deterministico del team ($1 \leq k \leq 254$).
2. **Topologia dei Container (`containers`):** Rappresenta la lista delle macchine che compongono la sfida. Per ciascun container vengono dichiarati:
   - `name`: Nome logico della macchina all'interno della challenge (ad esempio `web` o `database`).
   - `image`: Alias dell'immagine Incus da istanziare.
   - `ip_suffix`: Ultimo ottetto dell'indirizzo IP statico assegnato al container (`10.k.1.X`), dove `10.k.1.1` è riservato al gateway.
   - `limits_cpu` e `limits_memory`: Vincoli hardware imposti tramite cgroup.
3. **Runner di Verifica Effimero (`verifier`, opzionale):** Definisce un container di test dedicato, istanziato esclusivamente durante la fase di verifica all'interno della rete OVN (ad esempio con suffisso IP `250`).
4. **Comandi di Readiness (`readiness_commands`):** Serie di controlli preliminari eseguiti all'avvio della sandbox per accertarsi che i servizi interni abbiano completato la fase di bootstrap, prima di invocare i test veri e propri. Ciascun comando include intervalli di retry e policy di timeout.
5. **Comandi di Verifica (`verify_commands`):** Rappresentano la suite di convalida che certifica la patch. BlueAgent supporta due modalità operative distinte:
   - `target: container`: verifica in White Box eseguita direttamente all'interno dell'istanza bersaglio (ad esempio per verificare lo stato di un servizio con `systemctl`).
   - `target: verifier`: verifica in Black Box eseguita dal container tester per simulare l'attacco di rete o verificare l'integrità del servizio attraverso la LAN virtuale.

Esempio di Configurazione YAML: Di seguito è riportato un esempio sintetico di descrittore dichiarativo per una challenge multi-tier composta da un'applicazione web e un database:

```yaml
challenge_id: "patch-web-sqli"
verify_internet_access: true

containers:
  - name: "database"
    image: "debian-12-mysql"
    ip_suffix: 3
    limits_cpu: "1"
    limits_memory: "512MB"

  - name: "web"
    ...

verifier:
  image: "verifier-python"
  ...

readiness_commands:
  - target: "container"
    container_name: "database"
    cmd: ["mysqladmin", "ping", "-h", "127.0.0.1"]
    retry: 2
    timeout: 15

verify_commands:
  - target: "container"
    container_name: "web"
    cmd: ["systemctl", "is-active", "--quiet", "flask-app"]
    timeout: 5
  - target: "verifier"
    ...
```

Parsing e Risoluzione Dinamica degli Indirizzi IP nei Comandi: Poiché la specifica YAML viene scritta dall'autore della sfida in modo indipendente dallo specifico team, i comandi non possono contenere indirizzi IPv4 cablati staticamente. BlueAgent implementa un meccanismo di template interno:

- `{{ip:<name>}}`: viene tradotto nell'indirizzo IPv4 completo `10.<team_k>.1.<ip_suffix>` associato al container con nome logico `<name>`.

- `{{subnet}}`: viene sostituito con il prefisso di rete assegnato al team (`10.<team_k>.1`).
- `{{verifier_ip}}`: corrisponde all'IP assegnato al runner di test (`10.<team_k>.1.<ip_suffix>`).
- `{{team_k}}`: viene sostituito dall'identificatore numerico del team.

4.3.2 Configurazione delle Immagini con Distrobuilder

`Distrobuilder`: È lo strumento open source ufficiale per creare immagini di sistema per Incus ed LXC a partire da alberi di pacchetti e distribuzioni upstream ufficiali.

La configurazione di un'immagine in Distrobuilder avviene mediante un file dichiarativo. I campi principali sono i seguenti:

1. **Sorgente ed Immagine (`image` e `source`):** Indica quale immagine usare come base.
2. **Gestione dei Pacchetti (`packages`):** Definisce l'elenco dei software da installare all'interno del rootfs per preparare l'ambiente. È possibile anche rimuovere pacchetti preinstallati nell'immagine.
3. **Fasi di Hook e Predisposizione Rete (`actions`):** Consente di eseguire script durante le fasi intermedie di generazione del filesystem. È essenziale per predisporre le directory dei demoni di rete o specificare configurazioni interne al sistema.

All'interno dell'architettura di BlueAgent, il sottosistema `ImageService` invoca Distrobuilder per generare il tarball da importare nello storage pool ZFS.

Capitolo 5

Architettura nel Dettaglio e Validazione

5.1 Architettura del Codice

A livello software, BlueAgent organizza l'interazione con l'host disaccoppiando l'API in `/api` dalla gestione a basso livello dei container e delle componenti di rete, gestita ad alto livello dai servizi in `/services`, che contiene la logica di business, mentre le complessità operative sono delegate a moduli e driver specializzati (come `/engine`, `/network`, `/verifier`, `/images` e `/lifecycle`).

5.1.1 Astrazione del Runtime: `ContainerEngine`

BlueAgent non invoca direttamente il demone Incus all'interno della logica delle challenge. Tutte le operazioni di gestione del ciclo di vita dei container sono intermediate da un'interfaccia astratta denominata `ContainerEngine`.

Questa classe astratta formalizza esclusivamente le primitive necessarie ai requisiti operativi del Cyber Range:

- Creazione, avvio, arresto forzato ed eliminazione delle istanze.
- Duplicazione rapida e gestione delle interfacce di rete.
- Esecuzione dei comandi nei container.

Disaccoppiare l'interfaccia dall'implementazione permette ai servizi di non dover conoscere come avviene la gestione dei container, e di predisporre il codice per poter utilizzare altri gestori.

L'implementazione concreta `IncusEngine` comunica con il demone di sistema tramite Unix Domain Socket (`/var/lib/incus/unix.socket`), traducendo le primitive astratte in chiamate REST HTTP asincrone.

> `Unix Domain Socket (UDS)`: Meccanismo di Inter-Process Communication (IPC) gestito interamente dal kernel del sistema operativo. La comunicazione avviene trasferendo i dati direttamente tra i buffer di memoria dei processi nello spazio kernel senza attraversare lo stack di rete. L'endpoint è rappresentato da un file speciale sul filesystem (es: `/var/lib/incus/unix.socket`).[1]

5.1.2 Ciclo di Vita della Sandbox: Async Context Manager (`OVNVerifier`)

Il processo di verifica richiede di istanziare una replica fedele della rete di produzione che deve esistere esclusivamente per la durata dei test. In caso di qualsiasi tipo di fallimento, l'infrastruttura clonata deve essere rilasciata per evitare risorse allocate e sprecate.

Per garantire questa proprietà, il ciclo di vita della sandbox è incapsulato all'interno della classe `OVNVerifier`, implementata come **Asynchronous Context Manager** di Python.

> **Pattern RAII (Resource Acquisition Is Initialization):** L'allocazione delle risorse è vincolata dall'acquisizione del contesto, mentre il loro rilascio e smantellamento deterministico è garantito all'uscita dal blocco, gestendo automaticamente qualsiasi anomalia o interruzione. [2]

Il ciclo di vita dell' `OVNVerifier` segue tre fasi:

1. **Inizializzazione (`__aenter__`):**
   - Creazione della rete OVN temporanea
   - Duplicazione dei container tramite snapshot ZFS
   - Riconnessione dell'interfaccia `eth0` dei cloni alla nuova rete OVN
   - Istanziazione opzionale del container *Verifier Runner*.
2. **Esecuzione delle Verifiche (`run_checks`):**

[1] Linux manual page, unix(7) - Sockets for local interprocess communication: https://man7.org/linux/man-pages/man7/unix.7.html

[2] Bjarne Stroustrup, Resource Acquisition Is Initialization (RAII), C++ Programming Language.

- Esecuzione controllata dei comandi di test con timeout

3. **Rilascio e Teardown (`__aexit__`):**
    1. Arresto ed eliminazione del container `verify-runner`;
    2. Arresto forzato ed eliminazione di tutti i container cloni;
    3. Eliminazione della rete virtuale OVN.

L'uso dell'**Async Context Manager** fornisce dunque un metodo sicuro per utilizzare l'oggetto `OVNVerifier`.

5.1.3 Concorrenza e Two-Tier Locking

Incus non supporta operazioni concorrenti sul medesimo container. Inoltre per evitare problemi di accesso alle risorse è opportuno implementare un sistema di lock.

BlueAgent implementa un meccanismo di sincronizzazione a due livelli (`TwoTierLock`), dove sequenzialmente si richiede un tier dopo l'altro:

- **Tier 1 (Intra-processo):** utilizza `asyncio.Lock` per effettuare il lock delle risorse all'interno di un singolo processo.
- **Tier 2 (Inter-processo):** applica un lock non bloccante sul filesystem (`fcntl.flock`) condiviso tra più processi

Questo lock protegge l'accesso alle risorse in due casi:

- BlueAgent esegue su un solo processo (worker), ma le richieste potrebbero essere gestite da più task contemporaneamente. Allora il lock Tier 1 protegge l'accesso alla stessa risorsa da parte di due task sullo stesso worker.
- BlueAgent esegue su più processi (worker), allora il lock Tier 2 protegge l'accesso alla stessa risorsa da parte di due task su worker differenti.

  L'esecuzione multi-processo è abilitata configurando il numero di worker e viene gestito da uvicorn.

  `uvicorn`: è il web server ASGI ad alte prestazioni su cui si appoggia FastAPI; mentre FastAPI modella e valida le rotte dell'applicazione, Uvicorn governa l'accettazione delle connessioni HTTP e ne distribuisce il carico tra worker e task asincroni.

La risorsa principale su cui avviene la mutua esclusione nel codice è il team.

Il lock può essere effettuato anche sulla singola istanza, o a livello globale, utile quando si interagisce con meccanismi di sistema come `nftables`.

I lock inoltre sono rientranti, dunque uno stesso task può richiedere più volte lo stesso lock incrementando all'acquisizione il suo contatore.

5.1.4 Pipeline Asincrona non Bloccante su Socket Unix

BlueAgent utilizza il modello asincrono per comunicare con la socket Incus ed adattarsi al metodo `wait` che espone. La classe `IncusEngine` comunica con il socket UDS locale (`/var/lib/incus/unix.socket`) per mezzo di `httpx.AsyncClient`.

Successivamente se Incus gestisce la richiesta in background, BlueAgent interroga l'endpoint di Incus `/1.0/operations/<uuid>/wait` per sapere quando l'operazione viene rilasciata in modo asincrono.

Inoltre, l'esecuzione dei comandi sulla macchina host avviene tramite `asyncio.create_subprocess_exec` all'interno del runner asincrono, evitando chiamate bloccanti sul task.

Evitare il busy waiting (o comunque operazioni bloccanti) sui task uvicorn è essenziale in BlueAgent per mantenere il sistema reattivo nella gestione delle richieste che spesso possono richiedere vari secondi per essere soddisfatte.

5.1.5 Resilienza e Riconciliazione dello Stato (`LifecycleManager`)

Qualora l'host subisca un riavvio forzato improvviso o il processo di BlueAgent venga terminato bruscamente con `SIGKILL`, le routine di pulizia (`__aexit__`) non possono intervenire, lasciando container e switch virtuali orfani sul sistema.

Per risolvere queste anomalie BlueAgent ha un demone in background, il `LifecycleManager`. Ogni 600 secondi (10 minuti, o comunque un tempo configurabile) il demone interroga i metadati di Incus, per ogni container di verifica, ed elimina quelli con età superiore al **grace period** (per esempio, 600 secondi).

In questo modo si eliminano le risorse di verifica rimaste orfane per qualsiasi motivo.

5.1.6 Compilazione delle immagini

La generazione delle immagini prevede di calcolare lo **SHA-256** della configurazione ricevuta. Questo permette di gestire i seguenti casi:

- Se un'immagine con stesso hash e stesso id è già presente nel database allora il sistema non fa nulla.
- Se l'hash differisce ma l'id è presente nel database allora si può forzare la ricompilazione dell'immagine usando la nuova configurazione.

La compilazione delle immagini è protetta da un lock sul nome dell'immagine, ed il log relativo viene riversato su un file `.log` accessibile tramite `GET /images/{alias}/logs`.

5.2 Comandi per l'Architettura di Rete

Per andare a fondo nei dettagli implementativi e spiegare meglio alcuni concetti, sono riportati alcuni passaggi fondamentali nella configurazione della rete dell'host che esegue BlueAgent, o di comandi eseguiti direttamente da BlueAgent.

5.2.1 Configurazione OVN e Open vSwitch

Di seguito è riportata la configurazione per implementare la SDN. Il **piano dei dati** è implementato da Open vSwitch (**OVS**), mentre il **piano di controllo** è gestito da OVN, che comunica con Incus tramite il database Northbound.

Si configurano i parametri in OVS affinché l'agente locale `ovn-controller` possa comunicare con il database Southbound di OVN (`ovnsb_db.sock`):

```
sudo ovs-vsctl set open_vswitch . \
    external_ids:ovn-remote=unix:/run/ovn/ovnsb_db.sock \
    external_ids:ovn-encap-type=geneve \
    external_ids:ovn-encap-ip=127.0.0.1
```

- **`ovn-encap-type=geneve`**: Specifichiamo Geneve come protocollo da utilizzare per il tunneling. Questo passaggio è necessario anche se non utilizziamo mai Geneve
- **`ovn-encap-ip=127.0.0.1`**: Imposta la terminazione del tunnel nel localhost.

Si configura invece il database Northbound di OVN in modo che possa comunicare con Incus:

```
ovn-nbctl set-connection ptcp:6641:127.0.0.1
incus config set network.ovn.northbound_connection tcp:127.0.0.1:6641
```

OVN Northbound: memorizza la dichiarazione delle reti da creare e le proprietà che devono rispettare. **OVN Southbound:** memorizza i flussi logici compilati da `ovn-northd`, che `ovn-controller` traduce in regole OpenFlow concrete per il piano dei dati di OVS.

5.2.2 production-k Routing e Connettività dei Peer (WireGuard e nftables)

L'accesso dei partecipanti ai container della rete production-k richiede l'applicazione di regole di inoltro esplicite tra wg-k e br-team-k, da applicare nel kernel Linux mediante

`nftables`.

Le regole per il traffico vengono definite dentro la tabella `inet blueagent`:

```
sudo nft add table inet blueagent
sudo nft add set inet blueagent team_links '{ type ifname . ifname; flags interval; }'

sudo nft add element inet blueagent team_links { "wg-$k" . "br-team-$k", "br-team-$k" .
    "wg-$k" }
```

All'interno della catena `forward`, prima di accettare il traffico, viene forzato il *clamping* del MSS per il traffico TCP per prevenire blackhole dovuti alla riduzione di MTU dei tunnel. Subito dopo, il traffico lecito tra membri e container dello stesso team viene autorizzato verificando l'appartenenza al set `@team_links`:

```
sudo nft add chain inet blueagent forward '{ type filter hook forward priority filter;
    policy accept; }'
sudo nft add rule inet blueagent forward tcp flags syn / syn,rst tcp option maxseg size
    set rt mtu
sudo nft add rule inet blueagent forward iifname . oifname @team_links accept
```

A priori, si vuole impedire l'inoltro dei pacchetti applicando l'azione `drop` tra le interfacce `wg` e `br-team` non autorizzate, in modo da bloccare rotte e comunicazioni incrociate non esplicitamente accettate nella tabella:

```
sudo nft add rule inet blueagent forward iifname "wg-*" oifname "br-team-*" drop
sudo nft add rule inet blueagent forward iifname "br-team-*" oifname "wg-*" drop
sudo nft add rule inet blueagent forward iifname "br-team-*" oifname "br-team-*" drop
sudo nft add rule inet blueagent forward iifname "wg-*" oifname "wg-*" drop
sudo nft add rule inet blueagent forward iifname "br-team-*" accept
sudo nft add rule inet blueagent forward oifname "br-team-*" ct state established,related
    accept
```

Per consentire l'accesso a Internet ai container e ai peer, viene applicato il mascheramento dell'indirizzo sorgente. Si deve dunque abilitare il `nat` (il meccanismo di traduzione degli indirizzi) poco prima che il frame esca e venga immesso fisicamente sulla porta. Infine si specifica che si applica il nat dinamico (`masquerade`) per gli indirizzi che provengono dalla rete dei container verso l'esterno.

```
sudo nft add chain inet blueagent postrouting '{ type nat hook postrouting priority
    srcnat; policy accept; }'
sudo nft add rule inet blueagent postrouting ip saddr 10.0.0.0/8 ip daddr != 10.0.0.0/8
    masquerade
```

5.3 Plugin CTFd

CTFd è una delle piattaforme open source più diffuse per la gestione di competizioni Capture The Flag. Nativamente, CTFd è progettato per sfide di tipo *Jeopardy*, in cui la risoluzione coincide con l'invio di una stringa alfanumerica statica o regex (la *flag*).

Per abilitare gare di tipo *Defense* o *Patching*, in cui l'obiettivo del team consiste nel mettere in sicurezza un servizio vulnerabile mantenendone la piena operatività, è stata sviluppata un'estensione per CTFd composta da due moduli integrati: `patch_challenges` e `patch_flags`. Il plugin agisce da intermediario tra l'interfaccia web della competizione e le API REST esposte da BlueAgent.

> CTFd usa Flask al posto di FastAPI (adottato in BlueAgent) ed usa SQLAlchemy per gestire facilmente il modello dei dati utilizzato nel codice.

5.3.1 Componenti del Plugin: PatchChallenge e PatchFlag

L'integrazione sfrutta l'architettura a plugin di CTFd estendendone le classi base:

- **`PatchChallenge`**: definisce una nuova tipologia di sfida. Ha un modello dei dati per facilitare l'integrazione con il database di CTFd. Questo modello memorizza la specifica dichiarativa della topologia in formato JSON/YAML e l'insieme dei comandi di verifica da inoltrare a BlueAgent.
- **`PatchFlag`** (**`BaseFlag`**): invece di confrontare una stringa inserita dal giocatore con un valore memorizzato nel database, il metodo `compare()` interroga l'Oracolo (BlueAgent) e registra il tentativo di verifica ed il suo esito.

5.3.2 Distribuzione delle Configurazioni WireGuard

Per permettere ai partecipanti di operare sui container della propria challenge, il plugin gestisce automaticamente l'accesso alla rete del team:

1. Il plugin espone all'interfaccia web le rotte per la gestione della connettività.

2. Quando un utente autenticato visualizza la sfida o richiede l'accesso, CTFd invoca `PUT /teams/{team_k}/members/{user_id}` di BlueAgent.
3. BlueAgent alloca dinamicamente un indirizzo IP libero all'interno della subnet VPN del team (`10.k.0.0/24`), genera la coppia di chiavi asimmetriche e registra il peer sull'interfaccia WireGuard.
4. Il plugin restituisce al browser il file di configurazione WireGuard (`.conf`) pronto per l'importazione.

5.3.3 Flusso di Verifica e Convalida

Il ciclo di verifica e attribuzione del punteggio si articola nelle seguenti fasi:

1. **Richiesta di verifica**: il giocatore, dopo aver applicato le contromisure sui file e sui servizi del container, preme il pulsante di verifica sull'interfaccia di CTFd.
2. **Inoltro all'Oracolo**: CTFd invia una richiesta `POST /teams/{team_k}/verificator` a BlueAgent, trasmettendo l'identificativo del team e la specifica della challenge.
3. **Esecuzione**: BlueAgent esegue la verifica in ambiente separato.
4. **Aggiornamento punteggio**: BlueAgent restituisce il verdetto a CTFd che si occupa di aggiornare i punteggi.

5.4 Validazione

Illustreremo e commenteremo adesso i dati ottenuti testando BlueAgent. Per tutti i test è stato eseguito BlueAgent all'interno di una macchina virtuale Debian 13 con a disposizione 8 Core AMD Ryzen 7 7700X, 16GB RAM DDR5 5600MHz e il pool ZFS ospitato all'interno di un SSD. BlueAgent è stato avviato utilizzando solo un processo uvicorn che può all'occorrenza suddividere il lavoro in più task.

I test effettuati si occupano di fare richieste HTTP verso BlueAgent e di misurare il tempo richiesto per soddisfarle (ottenendo in risposta anche il tempo per l'esecuzione delle varie sottofasi). Le richieste sono organizzate in barriere:

- B_1: Provisioning per M team (creazione delle interfacce di rete).
- B_2: Deploy di N container per ogni team (creazione, iniezione configurazioni e avvio).
- B_3: Lancio della verifica per ogni team (creazione rete OVN e clonazione CoW).
- B_4: Teardown dell'ambiente di verifica (distruzione dei cloni e delle reti di test).
- B_5: Teardown delle macchine (arresto ed eliminazione dei container di produzione).

- B_6: Teardown delle infrastrutture dei team (rimozione delle reti OVN e dei bridge host).

Per assicurarsi sincronia nelle misurazioni effettuate, il test è stato organizzato in barriere $B_1, \ldots, B_6$. Per superare una barriera si attende che tutte le operazioni sulle risorse siano completate, così da evitare misurazioni errate.

> La sincronia è necessaria per evitare casi in cui qualche container debba essere ancora avviato (B_2) mentre altri team stanno già avviando la verifica (B_3), alterando i campionamenti.

Ad ogni barriera vengono eseguite una serie di richieste HTTP verso BlueAgent mentre ogni 100ms si campiona il contenuto di `/proc/stat` per ottenere:

- $\text{CPU}_{\text{util}} = \frac{\Delta(\text{user+system+nice})}{\Delta\text{total}} \times 100$
- $\text{IO}_{\text{wait}} = \frac{\Delta\text{iowait}}{\Delta\text{total}} \times 100$

I valori $\Delta\text{user}, \Delta\text{system}, \Delta\text{nice}, \Delta\text{iowait}$ sono presi campionando `/proc/stat`

- `nice`: tempo speso per i processi con bassa priorità
- `user`: tempo speso per eseguire i processi con priorità normale
- `system`: tempo speso nel kernel
- `iowait`: tempo in cui la CPU è in idle, in attesa di I/O.

> Il file virtuale `/proc/stat` espone il tempo speso dalla CPU nelle varie modalità operative. Campionando i contatori a intervalli discreti e calcolandone la differenza rispetto al tempo totale, CPU_{util} isola l'effettivo impegno di calcolo del processore, mentre IO_{wait} quantifica la frazione di tempo in cui la CPU è rimasta in stato di attesa attiva per il completamento di operazioni di I/O sul sottosistema di storage.

Per ottenere l'uso della RAM invece sono stati campionati `/proc/meminfo` e `/proc/spl/kstat/zfs/arcstat`
per ottenere:

- $\text{RAM}_{\text{usata_grezza}} = \text{MemTotal} - \text{MemFree} - \text{Buffers} - \text{Cached}$
- $\text{RAM}_{\text{netta_container}} = \text{RAM}_{\text{usata_grezza}} - \text{ARC}_{\text{size}} - \text{RAM}_{\text{baseline}}$
  - `Buffers`: è la memoria allocata dal kernel per conservare dati e metadati.
  - `Cached`: memoria impiegata per tenere in RAM i file.
  - `ARC`: è la Adaptive Replacement Cache di ZFS, memoria cache che si allarga per accelerare l'accesso al disco.

I test sono stati eseguiti utilizzando due immagini differenti in due run separate:

- `alpine:3.21` con una rootfs di 3.21MB.
- `ubuntu-heavy`: un'immagine `ubuntu:22.04` arricchita con pacchetti aggiuntivi come `systemd`, `nginx`, `sqlite3`, `python` e `postgresql` per renderla più pesante e "realistica". La rootfs per questa immagine è di $\sim$ 1.5GB.

Le barriere di sincronizzazione sono state inoltre valutate per diverse combinazioni di M e N, fino a raggiungere gli scenari più gravosi con $M = 10$ ed $N = 5$ (o viceversa).

> Il numero totale di container istanziati in un test è sempre $M \cdot N$. In fase di verifica il totale arriva a $2 \cdot M \cdot N$.

Un fattore importante da analizzare nei test è

$$\text{Speedup} = \frac{\text{Tempo di Deploy Create}}{\text{Tempo di CoW Clone}}$$

descrive il fattore moltiplicativo tra il tempo di creazione dell'istanza da 0 ed il tempo necessario per clonarla sfruttando i metadati Copy-on-Write di ZFS.

Successivamente ci riferiremo alla tupla (M, N) come al test eseguito con M team assegnando N container ad ognuno.

5.4.1 Confronto tra Alpine e Ubuntu

Nella Figura 5.1 (*Lifecycle phase timing comparison*) sono mostrati i tempi di esecuzione per tutte le barriere.

Come ci si poteva aspettare, presi singolarmente i casi $(5, 10)$ ed $(10, 5)$ hanno performance simili poiché condividono lo stesso numero complessivo di container. In particolare su tutti i test effettuati si ha che $T(B_3) < T(B_2)$: la fase di verifica è più rapida del deploy e, in entrambi questi scenari ad alta concorrenza, la differenza è di circa 20 secondi su Alpine e oltre 25 secondi su Ubuntu Heavy.

> La ragione di questo vantaggio risiede nella diversa metodologia usata: nella barriera B_2 (`Deploy Challenges`), Incus deve istanziare una macchina a partire dall'immagine di base presente nella pool ZFS, allocando nuove strutture nel sistema. Nella barriera B_3 (`Verify Setup`), invece, il container di produzione è già esistente e configurato: BlueAgent esegue uno snapshot atomico dello stato del container e invoca una clonazione Copy-on-Write.

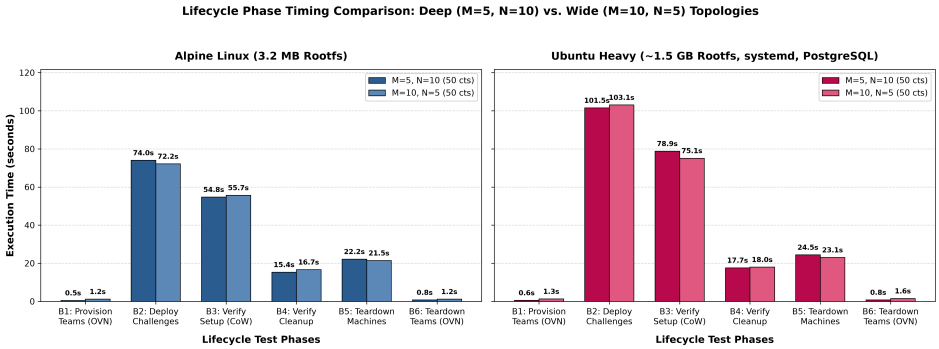


Figura 5.1: Confronto dei tempi di esecuzione delle macrofasi di benchmark

5.4.2 Confronto tra Clonazione e Deploy

Nei seguenti grafici la voce `Deploy Create` descrive il tempo medio richiesto da BlueAgent per creare tutte le istanze per un team, mentre `Verify CoW Clone` descrive il tempo medio richiesto da BlueAgent esclusivamente per clonare i container necessari alla verifica per un team.

> **Nota**: viene calcolato e mostrato nei grafici il tempo medio per un singolo team, ma bisogna tenere conto che le operazioni vengono eseguite contemporaneamente su più team tramite richieste concorrenti.

Dal grafico in Figura 5.2 (Graph 1), si nota che mentre il tempo per `Deploy Create` sale all'aumentare della concorrenza e della complessità dell'immagine, il tempo per `Verify CoW Clone` è nettamente più basso e dipende fortemente dai parametri M, N del test.

> Si riesce ad ottenere nei vari casi uno Speedup pari a 3.6 nel caso $(10, 5)$ con Alpine.

Entrambe le metriche sono strettamente confrontabili in quanto misurano l'intervallo necessario a portare un'istanza nello stato creato (spento), escludendo l'avvio e la configurazione di rete che vengono cronometrati in sotto-fasi successive.

In generale si nota che per $(5, 10)$ il tempo di clonazione è maggiore rispetto a $(10, 5)$. Questo perché internamente `uvicorn` usa 10 task differenti per gestire 10 richieste POST concorrenti (che logicamente provengono da 10 team differenti). Nel caso $(10, 5)$, ogni task si occupa di clonare in serie solo 5 container per il rispettivo team, sfruttando appieno il parallelismo dei core della CPU e accorciando la coda locale. L'altro caso $(5, 10)$ utilizza invece soltanto 5 task concorrenti, dove ciascun task deve gestire una catena di 10 container.

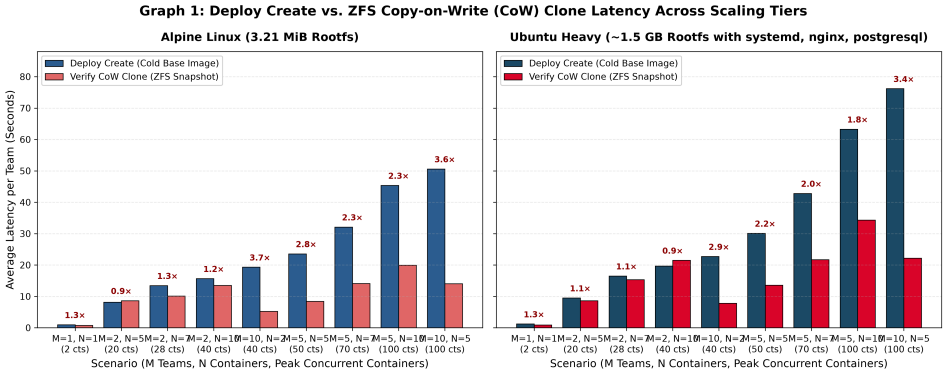


Figura 5.2: Confronto dei tempi tra clonazione CoW (ZFS) e deploy da catalogo

All'interno di un singolo task (associato ad un team), la clonazione dei container per quel team è sequenziale (prima A, poi B, poi C).

In generale, per migliorare ulteriormente le performance di clonazione in BlueAgent, si potrebbe teoricamente valutare di parallelizzare anche il processo di creazione del singolo team. Tuttavia un parallelismo eccessivo verrebbe limitato dalla gestione della mutua esclusione in ZFS e OVN.

Il diagramma in Figura 5.3 (Graph 2) mostra in media quanto impiega un team ad eseguire il deploy di una challenge e la verifica di una sandbox; il tempo viene ripartito nelle varie sotto-fasi coinvolte. In generale i tempi per clonare un container sono notevolmente inferiori rispetto ai tempi per creare un container da zero.

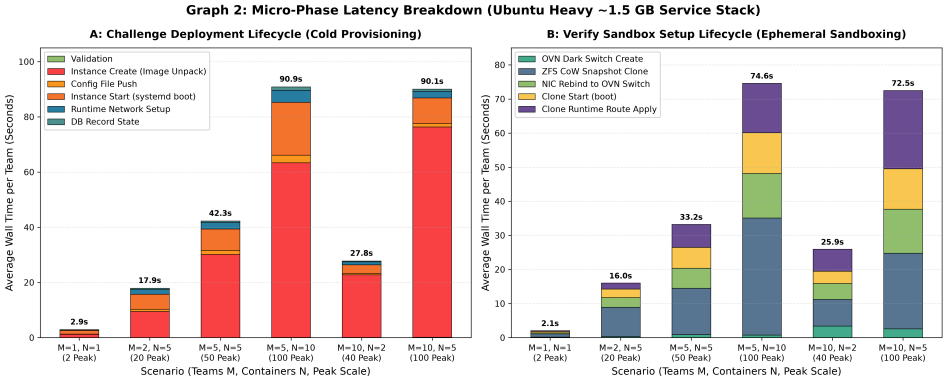


Figura 5.3: Scomposizione temporale delle sottofasi di verifica

Poiché la clonazione è nettamente più efficiente della creazione a partire dall'immagine, si potrebbe evolvere l'architettura implementando l'avvio preventivo di container template da clonare nella rete di produzione.

5.4.3 Uso di Memoria

Per farsi un'idea delle risorse RAM e CPU usate al variare del numero dei container sono riportati i grafici in Figura 5.4 e Figura 5.5. È importante notare che istanziare 100 container Ubuntu Heavy richiede un incremento netto di memoria di soli $\sim$ 5GB (circa 52.5MB per container a regime, contro i 15.3MB di Alpine). Cento container rappresentano una dimensione idonea a gestire una competizione Cyber Range di piccole dimensioni su un singolo server host.

Inoltre l'utilizzo medio della CPU durante le fasi di creazione non satura mai le risorse di calcolo dell'host, a riprova del fatto che la scalabilità nella gestione dei container è vincolata principalmente alle transazioni di I/O.

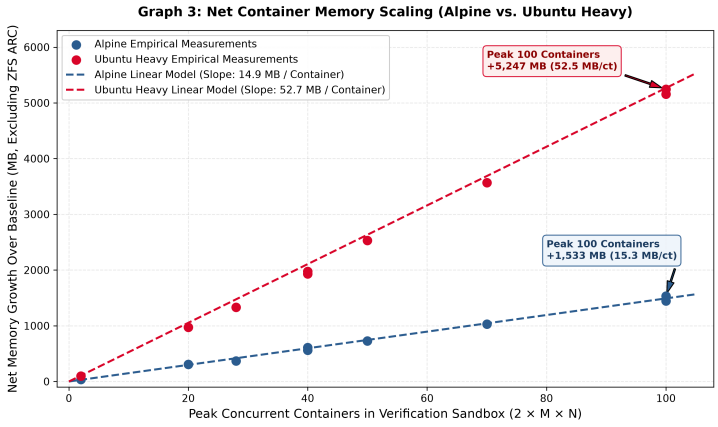


Figura 5.4: Andamento del consumo di memoria RAM netta al crescere dei container

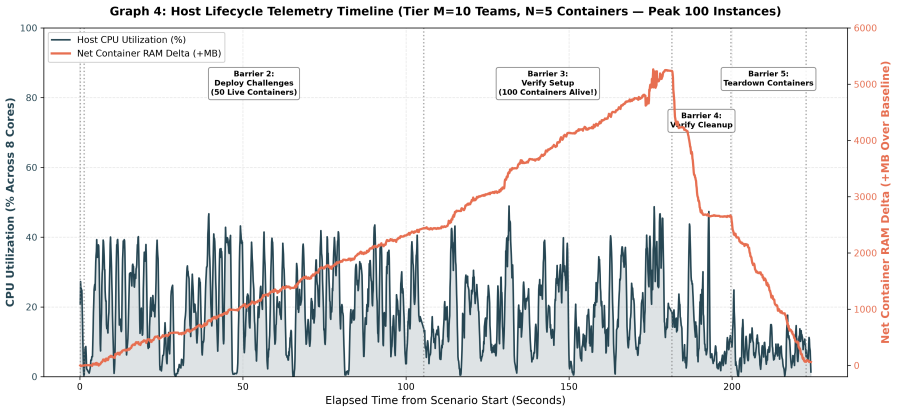


Figura 5.5: Timeline dettagliata del ciclo di vita e della concorrenza

Capitolo 6

Architetture di Rete Alternative

La sfida architetturale consiste nel duplicare l'infrastruttura di una challenge garantendo che i container clonati mantengano inalterati i loro indirizzi IP e la configurazione del gateway di default (per non alterare il comportamento dei payload dell'oracolo), senza generare conflitti di instradamento IP nell'host e garantendo la connettività Internet sia all'ambiente di verifica che di produzione.

Prima di convergere sulla soluzione adottata in BlueAgent sono state analizzate quattro architetture di rete alternative.

In questo capitolo vengono analizzate le caratteristiche tecniche, i vantaggi e i limiti operativi di ognuna.

6.1 Bridge L2 Senza Gateway

Questa architettura prevede la creazione di un bridge L2, per istanziare una rete LAN virtuale sui cui attaccare i container da verificare. I container duplicati mantengono la topologia di rete invariata, e possono comunicare tra di loro, tuttavia il bridge L2 non ha un gateway verso cui instradare per accedere alla WAN. Dunque al livello 3 i container non possono essere visti dall'esterno della LAN.

Questa architettura è molto semplice da implementare ed è anche efficiente, in quanto fa uso solo di strutture dati primitive del kernel Linux e di un unico bridge.

Tuttavia rende impossibile per i container l'instradamento verso l'esterno della rete. Dunque, dall'interno dei container di verifica, comandi come `apt` e `ping` verso host esterni falliranno.

Questa architettura è adatta per scenari di verifica che non richiedono accesso a Internet. Qualora tuttavia la connettività di rete esterna risulti necessaria per la validazione della challenge, tale approccio deve essere necessariamente scartato.

6.2 Interfaccia wan1 Aggiuntiva

Riprendendo l'architettura descritta nel paragrafo 6.1, nel tentativo di introdurre la connettività a Internet senza assegnare un indirizzo gateway L3 al bridge, si potrebbe valutare l'aggiunta ad ogni container duplicato di un'interfaccia secondaria `wan1` per l'inoltro verso `incusbr0`.

Il limite di questa soluzione risiede nella necessità di alterare le tabelle di routing interne ai container, introducendo regole che potrebbero interferire con le aspettative dell'oracolo di verifica o compromettere il comportamento del container.

Inoltre, questo approccio richiede di raddoppiare il numero complessivo di interfacce virtuali (poiché ciascuna `wan1` richiede una coppia di `veth` da collegare ad `incusbr0`). Tale meccanismo introduce complessità e compromette la fedeltà dell'ambiente di verifica, per cui è stato scartato.

6.3 Network Namespace Separato per la Verifica

In BlueAgent, abbiamo introdotto OVN per semplificare la gestione di due reti con stesso gateway.

Per evitare di aggiungere la complessità computazionale di OVN, si può pensare di creare un **network namespace** separato nell'host al momento della verifica.

All'interno di questo nuovo namespace, chiamato verify-k si crea la rete di verifica composta dal bridge L2 e si assegna a questo il gateway.

I container duplicati dovranno poi essere collegati al bridge nel namespace verify-k.

Per consentire l'accesso alla rete WAN bisogna usare una coppia di `veth`, per collegare il namespace verify-k al namespace root. In questo modo, i due bridge con stesso indirizzo IP possono coesistere poiché risiedono in namespace differenti: il kernel Linux effettua l'instradamento dei pacchetti verso le rispettive interfacce nei diversi namespace.

Tuttavia l'implementazione non è immediata e l'integrazione con Incus risulta problematica, in quanto l'orchestratore non prevede il tracciamento di interfacce di rete all'interno di altri network namespace. Bisogna aggirare tale vincolo, creando e gestendo direttamente all'interno di BlueAgent le coppie di *veth* necessarie a colmare questa discrepanza.

Nell'implementazione software aumenta la complessità: anziché delegare la gestione della rete all'API di Incus, è necessario invocare comandi di basso livello per gestire le *veth*.

6.4 Router Intermedio

Un'altra possibile alternativa consiste nell'introdurre un router dedicato (con un'immagine alpine per esempio) che fa da tramite tra il bridge di verifica e l'interfaccia `incusbr0`.

In questa architettura, i container duplicati vengono collegati a un bridge L2 isolato. Il router intermedio si aggancia al bridge usando come IP il gateway originale (`10.k.1.1/24`).

Verso l'esterno, il router ha una seconda interfaccia collegata verso `incusbr0` da cui ottiene un indirizzo IP. Vengono applicate le regole di SNAT per mascherare il traffico uscente. In questo modo la topologia interna della rete di verifica viene completamente nascosta, evitando i conflitti di instradamento con la rete di produzione.

Sul piano operativo l'adozione del router richiede di istanziare un container in più rispetto alla soluzione adottata con OVN.

In OVN inoltre la creazione dello switch, del router logico e delle regole di NAT è gestita in modo nativo da Incus, mentre l'uso di un router intermedio impone a BlueAgent di gestirlo manualmente per ogni sessione di verifica. Questo comporta un lieve aumento della complessità nel codice e del footprint di memoria.

6.5 Tabella riassuntiva

In sintesi, la seguente tabella riassume e pone a confronto le proprietà e i compromessi delle architetture analizzate.

Architettura	Fedeltà di Rete	Accesso WAN	Complessità Software	Overhead Risorse
6.1 Bridge L2 (senza GW)	Bassa (nessun default gateway)	Assente	Minima (bridge kernel)	Quasi nullo
6.2 Interfaccia wan1	Scadente (interfaccia anomala)	Presente (tramite wan1)	Media (routing custom container)	Basso
6.3 NetNS separato	Buona (topologia identica)	Presente (coppia veth)	Elevata (gestione manuale veth)	Basso
6.4 Router intermedio	Buona (topologia identica)	Presente (SNAT su router)	Medio-alta (container dedicato)	Medio (+1 container)
BlueAgent (OVN SDN)	**Buona (topologia identica)**	**Presente (SNAT a 2 stadi)**	**Minima (delegata a Incus)**	**Basso (condiviso)**

Capitolo 7

Conclusioni e Sviluppi Futuri

7.1 Difesa e Validazione di BlueAgent

Come Sistema Operativo su cui basare lo sviluppo è stato scelto **Debian 13**, in quanto offre un kernel stabile ed un'ottima compatibilità tra i vari pacchetti e le componenti utilizzate da BlueAgent, facilitando la configurazione di OVN, ZFS ed Incus.

Su questo sistema operativo si è scelto di utilizzare ZFS per creare lo storage pool per le immagini dei container. ZFS offre il meccanismo CoW a costo computazionale costante, ed è stato preferito rispetto ad alternative come BTRFS poiché ritenuto notevolmente più maturo e performante.

Si beneficia del CoW in due scenari chiave: quando si effettua il deploy della challenge per un team (clonando l'immagine base dallo storage pool) e durante la duplicazione per la verifica.

L'adozione di Incus è dovuta alla necessità di offrire ai concorrenti macchine realistiche mediante system container (dotati di un proprio init system completo e del processo `init` con `PID` 1). Tale approccio permette ai partecipanti di operare realisticamente accedendo a tutte le utility di sistema e installando pacchetti software. Incus gestisce inoltre nativamente gli storage pool ZFS e le reti OVN, semplificando l'architettura del software BlueAgent.

Il principale vantaggio introdotto da BlueAgent è l'invisibilità totale dell'Oracolo rispetto ai concorrenti: per i partecipanti è impossibile intercettare o sniffare il traffico di rete generato durante la validazione, poiché la rete di verifica verify-k risiede in un dominio SDN logicamente separato. In `nftables`, le regole di filtraggio consentono il transito unicamente tra l'endpoint VPN wg-k e il bridge di produzione br-team-k.

Inoltre, l'adozione del meccanismo CoW riduce significativamente i tempi di provisioning dei container come abbiamo visto in 5.4, mentre l'impiego di OVN automatizza

la configurazione della topologia di rete.

7.2 Compromessi e Sviluppi Futuri

Nonostante i vantaggi prestazionali e l'isolamento garantito dall'impiego congiunto di ZFS, Incus e OVN, l'architettura di BlueAgent presenta alcuni compromessi progettuali e delinea diverse opportunità di estensione futura.

7.2.1 Compromessi Architetturali

1. **MTU Blackhole per il traffico UDP:**
   Il traffico tra i partecipanti e i container transita attraverso il tunnel WireGuard, caratterizzato da una MTU di 1420 byte. Sebbene il problema della frammentazione sia risolto per il traffico TCP tramite il clamping dell'MSS in `nftables`, i pacchetti UDP non dispongono di un meccanismo di negoziazione analogo. Di conseguenza, datagrammi UDP generati con flag *Don't Fragment* (DF) e dimensione standard Ethernet (1500 byte) vengono scartati dal kernel senza recapito.

2. **Limite di scalabilità dello schema di indirizzamento:**
   La mappatura deterministica `10.k.x.x` associa l'identificatore del team k al secondo ottetto dell'indirizzo IPv4 ($1 \leq k \leq 254$). Questo vincolo delimita la capacità del sistema a un massimo di 254 team concorrenti per singola istanza, richiedendo una segmentazione a subnet più estese (o l'adozione di IPv6) per scenari di scala superiore.

3. **Confinamento del locking al singolo host:**
   Il sistema di sincronizzazione gerarchico `TwoTierLock` impiega `asyncio.Lock` a livello di processo e `fcntl.flock` a livello di filesystem locale. Questa soluzione garantisce consistenza su macchine multi-core mononodo, ma non è estendibile a cluster distribuiti. Scalare l'orchestrazione su più server fisici richiederebbe l'introduzione di un gestore di lock distribuito (es. Redis o etcd).

4. **Trade-off tra flessibilità e dipendenze esterne:**
   Per mantenere l'infrastruttura minimale e immediata nel deployment, si è scelto di non introdurre middleware complessi quali Kubernetes o Ansible. Tale approccio elimina complessità di configurazione, ma vincola la logica di rete delle challenge alle funzionalità esposte internamente da BlueAgent, rendendo meno agevole l'implementazione di tipi di challenge eterogenee.

7.2.2 Sviluppi Futuri

1. **Interfaccia da riga di comando (`bluecmd`):**
   Sviluppo di una CLI dedicata per fornire ad amministratori e creatori di challenge un tool ufficiale ed affidabile per gestire il ciclo di vita delle istanze.

2. **Monitoraggio dello SLA e Service Health Checking:**
   Integrazione di verifiche periodiche di disponibilità dei servizi di produzione. Il controllo continuo dello SLA (*Service Level Agreement*) è necessario in una competizione di Difesa per penalizzare chi cerca di barare spegnendo il servizio.

3. **Configurazione dichiarativa avanzata delle challenge:**
   Estensione del parser YAML per supportare scenari multi-tier complessi (es. router virtuali dedicati) e configurare per ogni container le interfacce di rete.

Capitolo 8

Appendice

8.1 Linux Networking

Per approfondire meglio il discorso relativo al routing dei pacchetti e delle interfacce di rete, si vuole elencare ad alto livello cosa succede all'interno del kernel linux e quali sono gli oggetti fondamentali.

8.1.1 Network Device Driver

Un network device opera sempre a livello 2 o 3 dello stack di rete, ed è rappresentato nel kernel dalla struttura `net_device`.[1] Questa struttura dati contiene varie informazioni:

- `IRQ`: numero identificativo del dispositivo
- `MTU`
- `MAC`: MAC address
- `Nome`: `eth0`, `enp0s3`, `wlan0`, ecc...
- `net_device_ops`: contiene i puntatori alle funzioni per la gestione del dispositivo (come le primitive start, stop, transmit, receive)

Un device driver si occupa di:

- **Ricevere** i pacchetti destinati al local host, inoltrandoli al livello di rete, e successivamente al livello di trasporto.
- **Trasmettere** i pacchetti generati dal local host, o fa il forward dei pacchetti ricevuti verso un'altra interfaccia.

[1] R. Rosen, *Linux Kernel Networking: Implementation and Theory*, Apress, 2014.

Ogni pacchetto in transito in tutti e due i sensi scatena un lookup nel *sottosistema di routing*, che decide cosa bisogna fare con quel pacchetto. La fase di routing si distingue in 5 fasi, detti **hook**, dove possiamo registrare dei **callback** per il **sottosistema netfilter**.

Per esempio l'hook `NF_INET_PRE_ROUTING`, registra la chiamata al callback per essere eseguito prima che il lookup del sottosistema di routing venga effettuato. Il callback ritorna un verdetto sul filtraggio del pacchetto, decidendo se bisogna accettarlo (`NF_ACCEPT`), oppure scartarlo (`NF_DROP`).

> Con `nftables` applichiamo le regole di filtraggio dei pacchetti in modo da intercettare e manipolare il sottosistema di routing
>
> Oltre ai callback registrati, altri fattori influenzano il modo in cui il routing viene effettuato. Per esempio un pacchetto viene scartato quando il TTL viene diminuito e arriva a 0.

La struttura `sk_buff` (Socket Buffer) definisce come un pacchetto nel kernel Linux viene rappresentato, tale struttura viene manipolata unicamente attraverso la SKB API.

La Socket Buffer viene allocata dinamicamente quando il pacchetto viene ricevuto sul cavo, ed è la struttura dati utilizzata per manipolare il pacchetto nel kernel sui vari layer di rete.

All'interno del kernel la struttura dati responsabile per il sottosistema di routing è la `fib_table`, ossia la tabella che raccoglie le rotte per il traffico destinato ad una certa sottorete.

8.1.2 Bridge

Un bridge è un dispositivo L2 che capisce il protocollo del livello di collegamento ed effettua l'inoltro dei dati copiandoli frame per frame.[2] Per farlo deve mantenere nel suo buffer almeno un frame per volta.

Un bridge assegna un indirizzo al livello di collegamento ad ognuna delle sue interfacce ed effettua il forward verso di queste se il bridge non è il destinatario del pacchetto. Opzionalmente può implementare il livello di rete (Layer 3) per offrire visibilità sulla WAN alle interfacce collegate. Un bridge viene di solito utilizzato per unire due LAN differenti, o per creare una LAN.

[2] C. Benvenuti, *Understanding Linux Network Internals*, O'Reilly Media, 2006.

Un bridge nel kernel Linux è un device virtuale, finché non viene collegato ad altri device non può né ricevere né trasmettere frame. Usa il concetto di enslaving: ossia gli vengono assegnate delle interfacce dette **enslaved** da controllare.

Quando si tenta di trasmettere su un bridge (es: `br0`), il device driver gestisce i device **enslaved** effettuando il forwarding dei frame.

8.1.3 Netfilter

È un sottosistema del kernel che fornisce le seguenti funzionalità all'interno del sottosistema di routing:[3] [4]

- Selezione e filtraggio dei pacchetti
- NAT
- Tracciamento delle connessioni
- Accounting di rete.

Per quanto riguarda il filtraggio dei pacchetti, abbiamo a disposizione 5 hook differenti:

- `NF_INET_PRE_ROUTING`: è chiamato nel metodo `ip_rcv()` del kernel, quindi ancora prima di entrare nel sottosistema di routing.
- `NF_INET_LOCAL_IN`: è chiamato nel momento in cui il pacchetto è indirizzato al local host.
- `NF_INET_FORWARD`: è chiamato dopo che è stato effettuato il lookup nel sottosistema di routing.
- `NF_INET_POST_ROUTING`: è chiamato per i pacchetti che devono essere inoltrati, pochi istanti prima che il pacchetto entri fisicamente nel cavo.
- `NF_INET_LOCAL_OUT`: è chiamato per i pacchetti creati nel local host, prima che escano.

Molte utility, comunemente usate, si basano sul sottosistema netfilter:

- `iptables`: agisce come front-end per netfilter ed implementa un **firewall**. Fornisce un'interfaccia per aggiungere, eliminare, regole netfilter, aggiungere tabelle, ecc...

[3] The Netfilter Project, *Netfilter Architecture and Packet Traversal Documentation*, https://www.netfilter.org/

[4] Netfilter Core Team, *nftables Documentation and HOWTO*, https://wiki.nftables.org/

- `nftables`: è sviluppato da **netfilter** (l'organizzazione dietro al sottosistema netfilter). È un rimpiazzo per iptables che è riconosciuto avere problemi di scalabilità e performance.

In `nftables`, utilizzato in BlueAgent, le regole sono organizzate in tabelle e catene: ad esempio, è stata definita una catena di tipo `nat` per gestire la traduzione degli indirizzi di rete.

Spesso le connessioni a livello applicativo hanno una connessione mantenuta nel tempo ed una serie di eventi che determinano la storia della comunicazione. Il **Connection Tracking** (`conntrack`) Layer permette al kernel di tenere traccia delle sessioni create, è la struttura dati fondamentale per implementare il NAT. `conntrack` mantiene per ogni connessione che deve usufruire del NAT le traduzioni necessarie per gli indirizzi IP.

8.2 WireGuard

La **VPN** è un servizio usato per comunicare in modo sicuro tra membri di un gruppo attraverso l'uso di una infrastruttura di comunicazione pubblica su cui il traffico viene incapsulato dentro ad un tunnel cifrato. [5]

Wireguard è una VPN che utilizza il concetto di **cryptokey routing**: viene mantenuta in memoria una tabella dove ad ogni chiave pubblica di un peer, viene associata una lista di indirizzi ip identificati come allowed.[6] Fornisce un'interfaccia stateless per utilizzare il tunnel VPN, ossia un'interfaccia di rete virtuale, che potrebbe essere per esempio `wg0`.

> Per **stateless**, si intende il fatto che l'amministratore non deve occuparsi della gestione esplicita dello stato di sessione, dell'handshake periodico o della riconnessione, poiché tali compiti sono demandati automaticamente al modulo kernel di WireGuard.

Rispetto ad OpenVPN, WireGuard è considerato superiore sul piano della sicurezza e dell'efficienza:

- OpenVPN supporta una vasta gamma di cifrari e opzioni TLS che incrementano notevolmente la superficie di attacco complessiva.

[5] M. S. et al., *Virtual Private Network (VPN) Protocols and Architecture*, IEEE, https://ieeexplore.ieee.org/document/4756450

[6] J. A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*, NDSS Symposium, 2017. https://www.wireguard.com/papers/wireguard.pdf

- OpenVPN opera nello spazio utente tramite dispositivi virtuali TUN/TAP, introducendo un overhead significativo dovuto a molteplici passaggi di contesto e copie di buffer di memoria tra *user space* e *kernel space*.

WireGuard non implementa un protocollo applicativo per lo scambio e la negoziazione iniziale delle chiavi, delegando tale incombenza a canali di distribuzione sicuri esterni e preesistenti (quali ad esempio orchestratori, agenti software dedicati o canali SSH protetti).

La struttura concettuale di una *cryptokey routing table* associa a ciascun peer autorizzato la relativa chiave pubblica crittografica e la lista di indirizzi IP sorgente consentiti:

Peer Public Key	Allowed Source IPs	Endpoint (Opzionale)	Descrizione
`xTIB...p8Dg`	`10.k.0.2/32`	`198.51.100.1:51820`	Giocatore 1 (Team k)
`TrMv...WXX0`	`10.k.0.3/32`	`203.0.113.2:51820`	Giocatore 2 (Team k)
`gN65...z6EA`	`10.k.0.4/32`	`198.51.100.5:51820`	Giocatore 3 (Team k)

La tabella determina quali coppie (Peer Public Key, Allowed Source IP) abbiano facoltà di inviare e ricevere pacchetti attraverso l'interfaccia virtuale `wg-k`.

Opzionalmente è possibile specificare verso quale endpoint inoltrare il pacchetto. L'**endpoint** identifica l'indirizzo IP pubblico e la porta UDP su cui il demone o modulo WireGuard remoto è in ascolto.

Il flusso di elaborazione e instradamento dei pacchetti si articola nelle seguenti fasi:

- l'interfaccia `wg0` riceve un pacchetto IP da inoltrare nel tunnel;
- il kernel incapsula il pacchetto all'interno di un datagramma UDP il cui payload viene autenticato e cifrato mediante il cifrario simmetrico `ChaCha20Poly1305`;
- il pacchetto cifrato viene trasmesso verso l'endpoint specificato, oppure verso l'indirizzo remoto appreso dinamicamente in occasione dell'ultimo handshake;
- il nodo ricevente valida la corrispondenza dell'indirizzo sorgente nella *cryptokey routing table*;
- il payload viene decifrato e autenticato, estraendo il pacchetto originale;
- il pacchetto in chiaro viene immesso nello stack di rete locale attraverso l'interfaccia virtuale wg0.

8.3 ZFS

In ZFS si segue questo principio base: *Un blocco in uso non viene mai sovrascritto.*[7]

[7]OpenZFS Documentation Project, *OpenZFS System Administration and Architecture*, https://openzfs.github.io/openzfs-docs/

Modificare dati vuol dire dunque scrivere il contenuto in una nuova porzione di memoria, ed il filesystem dovrà essere modificato per puntare a questa nuova area.

Questo principio, permette di implementare e gestire facilmente feature avanzate come snapshots ed il self-healing dei dati.

In ZFS lo storage è organizzato all'interno di un pool (denominato *zpool*), strutturato come un albero di blocchi:

- le foglie contengono i blocchi di dati effettivi;
- i nodi intermedi sono puntatori indiretti a blocchi (**block pointers**);
- l'**uberblock** è il nodo radice dell'albero, da cui è navigabile l'intero filesystem.

Un block pointer tiene traccia di un insieme di blocchi memorizzandone dimensione e **checksum**. Ciascun nodo intermedio valida crittograficamente i propri figli. Di conseguenza, modificare dati nel filesystem comporta il ricalcolo delle checksum e la riscrittura della catena dei nodi padri a ritroso fino alla radice, allocando nuovi blocchi per ogni livello.

Al termine della catena di modifiche, viene generato un nuovo uberblock', che fa riferimento al nuovo albero con i dati aggiornati e i puntatori consistenti. L'intera transizione avviene in modo **atomico**.

Il costo computazionale per una modifica è proporzionale all'altezza dell'albero, non alla dimensione assoluta dello storage pool. Questa proprietà permette di mantenere contemporaneamente valido sia il nuovo uberblock' sia il precedente uberblock.

ZFS segue un modello transazionale: le operazioni di scrittura vengono aggregate in gruppi di transazione (*transaction groups*, `txg`) applicati atomicamente su disco.

8.3.1 Vantaggi di Pool e transazioni

Gli snapshot del filesystem vengono eseguiti con complessità temporale $O(1)$: creare uno snapshot consiste semplicemente nell'impedire che i blocchi foglia dell'albero al momento dello snapshot vengano deallocati o sovrascritti. Per preservare l'intero stato del filesystem è sufficiente conservare il riferimento all'uberblock congelato.

> Uno snapshot in ZFS è una vista read-only del dataset congelata a un istante preciso. La sua creazione è un'operazione atomica, istantanea e a costo zero di spazio disco iniziale.

Integrità dei dati e self-healing: ZFS rileva corruzioni silenti confrontando le checksum memorizzate nei puntatori padri con il contenuto effettivo dei blocchi. Qualora il pool sia configurato con ridondanza (mirroring o RAID-Z), il filesystem è in grado di correggere automaticamente il dato corrotto attingendo dalla copia integra.

8.3.2 Considerazioni di Efficienza e Frammentazione

La natura Copy-on-Write implica che l'aggiornamento di dati preesistenti allochi blocchi in posizioni non contigue dello storage pool, frammentando progressivamente la sequenzialità fisica dei dati su disco (*fragmentation*). L'efficienza dell'allocatore di ZFS dipende pertanto dalla percentuale di spazio libero residuo nel pool e dal livello di frammentazione complessivo.

Inoltre, qualsiasi operazione di scrittura (inclusa la cancellazione di file) richiede l'allocazione di nuovi metadati. Nel caso di scritture parziali e casuali di piccole dimensioni, il costo di riallocazione dei metadati e il ricalcolo delle checksum possono risultare significativi.

8.4 NsJail e Container

Si vuole in questo capitolo approfondire i meccanismi utilizzati da NsJail per l'isolamento delle risorse del kernel Linux.[8] Tutti questi meccanismi utilizzati da NsJail sono utilizzati anche da Docker e Incus per implementare i container e verrà visto brevemente in che modo.

8.4.1 Namespaces

Un namespace è una feature del kernel Linux che permette di partizionare le risorse del kernel in modo che:[9]

- un insieme di processi è in grado di vedere un certo insieme di risorse
- un altro insieme di processi è in grado di vedere un altro insieme di risorse.

Il kernel Linux supporta diversi tipi di namespace: Cgroup, IPC, Network, Mount, PID, Time, User ed UTS.

Il **PID namespace** permette di partizionare gli ID dei processi in modo che per esempio, su due PID namespace differenti possano coesistere due processi con process ID identici. Inoltre un processo che esegue sul pid namespace A non può vedere quali processi sono in esecuzione sul pid namespace B.

Il **Network namespace** invece offre l'isolamento delle risorse di networking, come device, lo stack IPv4, le tabelle di routing, ecc. Un device fisico vive all'interno di

[8] Google Security Team, *NsJail Process Isolation Tool*, https://nsjail.dev/

[9] Linux manual page, *namespaces(7) - Overview of Linux namespaces*, https://man7.org/linux/man-pages/man7/namespaces.7.html

un unico network namespace, e le connessioni tra namespace sono gestite dai device `veth`.[10]

In sintesi, i container sfruttano i namespace per creare un ambiente isolato in cui eseguire i processi, montare il filesystem e configurare le interfacce di rete.

8.4.2 Seccomp

Seccomp permette a un processo di specificare un filtro per le chiamate di sistema.[11] Il filtro è espresso come un programma `cBPF` (classic BPF), che opera sul numero della syscall e sui suoi argomenti. Fornisce un meccanismo essenziale per minimizzare la superficie di attacco verso il kernel.

A titolo esemplificativo, l'utilizzo dell'API `libseccomp` in linguaggio `C` prevede la seguente sequenza operativa:

- Si inizializza nel codice il contesto del filtro: `scmp_filter_ctx ctx`
- Si aggiungono le regole selettive invocando `seccomp_rule_add(ctx, ...)`
- Si caricano le regole nel kernel con `seccomp_load(ctx, ...)`. Da questo istante, il processo e i suoi discendenti saranno soggetti ai vincoli definiti.

I container Docker tradizionali applicano Seccomp per bloccare le chiamate di sistema considerate pericolose, mentre Incus nei system container privilegiati o semi-privilegiati adotta una policy permissiva delegando l'isolamento agli user namespace.

8.4.3 cgroups

I Control Groups costituiscono un meccanismo per aggregare o partizionare insiemi di task e i loro discendenti in gerarchie controllate.[12]

In particolare, la documentazione del kernel Linux formalizza i seguenti concetti:

- `cgroup`: associa un insieme di task a un insieme di parametri e limiti per uno o più sottosistemi.
- `sottosistema`: noto anche come *resource controller*, schedula una risorsa hardware o applica quote e vincoli a un determinato raggruppamento di task.

[10] Linux manual page, *network_namespaces(7) - Network namespaces*, https://man7.org/linux/man-pages/man7/network_namespaces.7.html

[11] Linux manual page, *seccomp(2) - Operate on Secure Computing state of the process*, https://man7.org/linux/man-pages/man2/seccomp.2.html

[12] The Linux Kernel Organization, *Control Groups (cgroups) Documentation*, https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v1/cgroups.html

Tramite un cgroup è possibile ad esempio limitare la quota di CPU o di memoria RAM assegnata a un gruppo di task. Nei container, i cgroup sono utilizzati sistematicamente per imporre tetti massimi all'utilizzo di risorse computazionali e condurre attività di accounting e monitoraggio.

Bibliografia

[1] Google Security Team, "kCTF: Kubernetes-based Infrastructure for Capture The Flag Competitions," https://google.github.io/kctf/, 2020, accessed: 2026-10-12.

[2] ——, "kCTF Security Threat Model and Sandbox Architecture," https://github.com/google/kctf/blob/v1/docs/security-threat-model.md, 2021, accessed: 2026-10-12.

[3] G. Antoniak and Google Security Team, "NsJail: A Light Approach to Process Isolation on Linux Using Namespaces and Seccomp-BPF," https://github.com/google/nsjail/, 2016, accessed: 2026-10-12.

[4] Cloud Native Computing Foundation, "Kubernetes: Production-Grade Container Orchestration," https://kubernetes.io/, 2024, accessed: 2026-10-12.

[5] Domy, "CTFBox: Open-Source Attack-Defense CTF Platform," https://ctfbox.domy.sh/, 2023, accessed: 2026-10-12.

[6] C.-K. Pham, R. Beuran, and Y. Shinoda, "CyTrONE: An Integrated Cybersecurity Training Framework," *Computers & Security*, vol. 78, pp. 218–238, 2018.

[7] Cybersecurity Research and Development Group, JAIST, "CyTrONE: Cybersecurity Training Organizer and Network Environment," https://github.com/crond-jaist/cytrone, 2018, accessed: 2026-10-12.

[8] Masaryk University, "KYPO Cyber Range Platform and CyberRangeCZ Architecture," https://docs.platform.cyberrange.cz/, 2022, accessed: 2026-10-12.

[9] Air & Space Forces Association, "CyberPatriot: The National Youth Cyber Education Program," https://www.uscyberpatriot.org/, 2024, accessed: 2026-10-12.

[10] M. M. Yamin, B. Katt, and M. Nowostawski, "Serious Games and Cyber Ranges: A Survey of Cyber Ranges and Testbeds for Cybersecurity Education, Training, and Testing," *Sensors*, vol. 20, no. 24, p. 7148, 2020.

[11] M. Parisi *et al.*, "Current Status and Capabilities of SAST Tools in Software Security," *Advances in Emerging Trends with Science and Emerging Technologies (AETIC)*, vol. 4, no. 3, pp. 1–12, 2020.

[12] D. Breuker, "pspy: Monitor Linux Processes Without Root Permissions," https://github.com/DominicBreuker/pspy, 2021, accessed: 2026-10-12.

[13] CTFd LLC, "CTFd: The Open-Source Capture The Flag Platform," https://ctfd.io/, 2024, accessed: 2026-10-12.

[14] Linux Containers Project, "Incus: Powerful System Container and Virtual Machine Manager," https://linuxcontainers.org/incus/docs/main/, 2024, accessed: 2026-10-12.

[15] ——, "Incus Documentation: OVN Network Driver and Overlay Topology," https://linuxcontainers.org/incus/docs/main/reference/network_ovn/, 2024, accessed: 2026-10-12.

[16] J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[17] J. Gross, I. Ganga, T. Sridhar *et al.*, "Geneve: Generic Network Virtualization Encapsulation," IETF, RFC 8926, 2020.

[18] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, "The Design and Implementation of Open vSwitch," in *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 117–130.

[19] Netfilter Project, "nftables Documentation: Forwarding Traffic and TCP MSS Clamping," https://wiki.nftables.org/wiki-nftables/index.php/Forwarding_traffic, 2023, accessed: 2026-10-12.

[20] ——, "nftables Documentation: Performing Network Address Translation (NAT)," https://wiki.nftables.org/wiki-nftables/index.php/Performing_Network_Address_Translation_(NAT), 2023, accessed: 2026-10-12.

[21] G. of Geeks, "Virtual Ethernet Devices (veth) and Network Namespaces in Linux," https://dev.to/godofgeeks/virtual-ethernet-devices-veth-ofd, 2022, accessed: 2026-10-12.

[22] M. Kerrisk, "unix(7) — Linux Programmer's Manual: Sockets for Local Interprocess Communication," https://man7.org/linux/man-pages/man7/unix.7.html, 2021, linux man-pages 5.10.

[23] R. Rosen, *Linux Kernel Networking: Implementation and Theory*. Apress, 2014.

[24] C. Benvenuti, *Understanding Linux Network Internals*. O'Reilly Media, 2006.

[25] OpenZFS Project, "OpenZFS Documentation: Architecture, Datasets, Snapshots and Copy-on-Write," https://openzfs.github.io/openzfs-docs/, 2024, accessed: 2026-10-12.

[26] J. Ellingwood, "A Deep Dive into Iptables and Netfilter Architecture," https://www.digitalocean.com/community/tutorials/a-deep-dive-into-iptables-and-netfilter-architecture, 2015, digitalOcean Community Tutorial.

[27] S. Ramírez, "FastAPI: Modern, High-Performance Web Framework for Python," https://fastapi.tiangolo.com/, 2024, accessed: 2026-10-12.

[28] Encode, "Uvicorn: The Lightning-Fast ASGI Server," https://www.uvicorn.org/, 2024, accessed: 2026-10-12.

[29] Linux Containers Project, "Distrobuilder: System Container Image Building Tool," https://linuxcontainers.org/distrobuilder/, 2024, accessed: 2026-10-12.

[30] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013, resource Acquisition Is Initialization (RAII).

[31] B. Gleeson, A. Lin, J. Heinanen, G. Armitage, and A. Malis, "A Framework for IP Based Virtual Private Networks," *IEEE Communications Magazine*, vol. 38, no. 2, pp. 58–66, 2000.

[32] M. Kerrisk, "memfd_create(2) — Linux Programmer's Manual: Create an anonymous file in RAM," https://man7.org/linux/man-pages/man2/memfd_create.2.html, 2021, linux man-pages.

[33] ——, "fexecve(3) — Linux Programmer's Manual: Execute program specified via file descriptor," https://man7.org/linux/man-pages/man3/fexecve.3.html, 2021, linux man-pages.

Ringraziamenti

La tua direzione la dà il capo quando sbava
L'arte insegue il mercato, ma non vede la strada
Io cerco le regole e dei modi per infrangerle
Sentirmi così intensamente solo mi fa piange'
Ho dei pensieri strani: a lei farei del male fisico
Per farle sentire cosa ho dentro e quanto tanto mi spaventa
Oh, adesso ti impressioni? Il mio dolore è un eufemismo
Non lo sai quello che sento, però lo vorrei tantissimo

Nayt - guerra dentro

Ringrazio Gemini 4, Gemini 3.1 Pro, Gemini 3.8 Flash ed Antigraivity.